HTML Interview Questions – Detailed Answers

Basic Level

1. What is HTML and what does it stand for?

HTML stands for **HyperText Markup Language**. It is the standard markup language used to create and structure web pages. HTML describes the content of a page using a system of tags and attributes, defining elements such as headings, paragraphs, links, images, forms, and more. It forms the foundation of web development, which can then be styled with CSS and made interactive with JavaScript.

2. What is the purpose of `<!DOCTYPE html>`?

The `<!DOCTYPE html>` declaration informs the web browser which version of HTML the page is written in. In HTML5, it is a simple declaration that ensures the browser renders the page in **standards mode**, following modern web specifications. It must be the very first line in an HTML document, before the `<html>` tag. Without it, browsers may switch to **quirks mode**, emulating old, non-standard rendering, which can cause inconsistent layout and behavior.

3. What is the difference between HTML elements and HTML tags?

- **HTML tag** – The markup that defines the start or end of an element, enclosed in angle brackets (e.g., `<p>`, `</p>`).

- **HTML element** – The complete structure consisting of an opening tag, its content, and a closing tag (if applicable). For example, `<p>This is a paragraph.</p>` is an element; the `<p>` and `</p>` are tags.
  In short, **tags** are the building blocks, while **elements** are the complete units.

4. What is the difference between `<div>` and `<span>`?

- **`<div>`** – A **block-level** container used to group larger sections of content. It starts on a new line and takes the full width available. It is typically used for layout and styling.
- **`<span>`** – An **inline** container used to group small pieces of text or other inline elements within a line. It does not cause line breaks and only takes up as much width as necessary.
  Both are generic, non-semantic elements, but they serve different layout purposes.

5. What are semantic HTML elements? Give 5 examples.

Semantic HTML elements are tags that clearly describe their meaning and the type of content they contain, both to humans and machines (browsers, search engines, screen readers).
Examples: - `<article>` - `<section>` - `<nav>` - `<header>` - `<footer>`
Other examples: `<main>`, `<aside>`, `<figure>`, `<mark>`. Using semantic elements improves accessibility, SEO, and code maintainability.

6. What is the difference between <section> and <article>?

- **<section>** – Represents a thematic grouping of content, typically with a heading. It is a generic container for related content, such as chapters, tabbed content, or parts of a page.
- **<article>** – Represents a self-contained composition that could be distributed independently (e.g., a blog post, news story, forum post).
  An <article> can contain multiple <section>s, and a <section> can contain multiple <article>s. The key distinction is that the content of an <article> should make sense on its own.

7. When should you use <header>, <main>, and <footer>?

- **<header>** – Contains introductory content or navigational aids for its nearest ancestor. It can appear at the top of the page or within sections/articles, often holding headings, logos, or search forms.
- **<main>** – Represents the dominant content of the <body>. It should be unique to the document and not repeated across pages (e.g., sidebars, navigation). There must be only one <main> per page.
- **<footer>** – Contains information about its section, such as author, copyright, links, or related documents. It can be used for the whole page or inside sections/articles.

8. What is the purpose of the <nav> element?

The <nav> element defines a section of the page intended for navigation links, either within the current document or to external pages. It is used for major navigation blocks (menus, tables of contents) and helps screen readers and search engines quickly identify the primary navigation area.

9. What is the difference between block-level and inline elements?

- **Block-level elements** – Occupy the full width available, start on a new line, and stack vertically. They can contain other block or inline elements. Examples: , <h1>, <p>, <section>.
- **Inline elements** – Occupy only as much width as necessary, do not start on a new line, and flow within text. They can contain only data and other inline elements. Examples: , <a>, <img>, .
  The display behavior can be altered with CSS (e.g., display: block or inline).

10. What are void elements or self-closing tags in HTML? Give examples.

Void elements (also called empty or self-closing tags) are HTML elements that cannot have any content. They do not have a closing tag. In HTML5, they are written as a single tag, optionally with a trailing slash (e.g., <br> or <br />).
Examples: <br>, <img>, <input>, <hr>, <meta>, <link>.

Intermediate Level

11. What is the difference between `id` and `class` attributes?

- **`id`** – A unique identifier for a single element. No two elements on the same page may share the same `id`. It is used for CSS styling, JavaScript (`getElementById`), and anchor links.
- **`class`** – A non-unique identifier that can be used on multiple elements. It is used to apply common styles or behaviors. An element can have multiple classes (space-separated).
  `id` is specific and unique; `class` is reusable and general.

12. What are data attributes (`data-*`) and when would you use them?

Data attributes are custom attributes that store extra information on HTML elements. They are prefixed with `data-` and can be named anything (e.g., `data-user-id`, `data-toggle`). They are purely for metadata and do not affect rendering.
Use them to embed hidden data that can be accessed via JavaScript (using the `dataset` property) for dynamic interactions, such as storing configuration values, identifiers for AJAX calls, or state for UI components.

13. What is the purpose of the `alt` attribute in images?

The `alt` attribute provides alternative text for an image. It is displayed if the image fails to load and is read by screen readers for visually impaired users, making it essential for accessibility. Search engines also use it to understand image content. For purely decorative images, use `alt=""` (empty) to avoid distracting assistive technology.

14. What is the difference between `<strong>` and `<b>`, `<em>` and `<i>`?

- **`<strong>`** – Indicates strong importance or urgency. It is semantic and is typically rendered as bold.
- **`<b>`** – Only applies bold styling without any semantic meaning. It is presentational.
- **`<em>`** – Denotes emphasis, usually rendered as italic. It conveys stress or importance.
- **`<i>`** – Represents italic text for stylistic purposes (e.g., foreign words, technical terms) without added emphasis.
  Use semantic tags (`<strong>`, `<em>`) for meaning and CSS for purely visual styling.

15. What are meta tags and why are they important?

Meta tags are HTML elements placed in the `<head>` section that provide metadata about the document. They define information such as character set, description, keywords, author, viewport settings, and more. They are important because they: - Help search engines index and display pages (SEO). - Control browser behavior (e.g., `viewport` for responsive design, `charset` for encoding). - Enable social media platforms to display rich previews (Open Graph tags).
Common meta tags: `<meta charset="UTF-8">`, `<meta name="description" content="...">`, `<meta name="viewport" content="width=device-width, initial-scale=1.0">`.

16. What is the viewport meta tag and why is it crucial for responsive design?

The **viewport meta tag** controls how a webpage is displayed on mobile devices. It sets the width and scaling of the viewport. For example:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This tells the browser to set the viewport width to the device's width and use an initial zoom level of 1. Without it, mobile browsers may render the page at a desktop width and then shrink it, making text tiny and requiring zooming. It is essential for responsive design to ensure pages are readable and properly scaled across all devices.

17. What is the difference between `<link>` and `<script>` tags?

- **`<link>`** – Used to link external resources, most commonly CSS stylesheets. It is placed in the `<head>` and has attributes like `rel`, `href`, and `type`. Example: `<link rel="stylesheet" href="styles.css">`.
- **`<script>`** – Used to embed or reference JavaScript code. It can be placed in the `<head>` or `<body>`. It can contain inline code or link to an external file via the `src` attribute. Example: `<script src="script.js"></script>`.
  `<link>` is primarily for stylesheets and other resource linking; `<script>` is for scripting.

18. What are the different input types in HTML5? List at least 8.

HTML5 introduced several new input types to improve form handling. At least eight examples: - `text` - `email` - `password` - `number` - `date` - `checkbox` - `radio` - `file` - `range` - `color` - `tel` - `url` - `search`

19. What is the purpose of the `name` attribute in form inputs?

The `name` attribute identifies the form control when the form is submitted. The browser sends the **name-value pair** of each input to the server. The `name` becomes the key in the submitted data (e.g., in the query string for GET or in the POST body). It is also used by JavaScript to reference elements (e.g., `getElementsByName()`). It is essential for server-side processing to know what each input represents.

20. What is the difference between GET and POST methods in forms?

- **GET** – Appends form data to the URL as a query string. It is suitable for non-sensitive, idempotent requests (e.g., search forms). Data is visible in the URL, limited in length, and can be bookmarked.
- **POST** – Sends form data in the HTTP request body. It is more secure for sensitive information (though still requires HTTPS), has no size limits, and can handle file uploads. It is used for actions that change server state (e.g., submitting registration). Data is not visible in the URL and cannot be bookmarked.

21. What is the purpose of `<label>` and how should it be associated with inputs?

The `<label>` element defines a label for a form control, improving accessibility and usability. Clicking the label focuses/activates the associated input (especially useful for checkboxes and radio buttons). Association can be done in two ways: - **Implicit** – Wrap the input inside the label: `<label>Name: <input type="text"></label>`. - **Explicit** – Use the `for` attribute matching the input's `id`: `<label for="name">Name:</label> <input type="text" id="name">`.
Explicit association is often preferred for layout flexibility.

22. What are `required`, `pattern`, `min`, and `max` attributes used for?

These are HTML5 form validation attributes: - **`required`** – Specifies that the input must be filled out before submission. - **`pattern`** – Defines a regular expression the input's value must match (e.g., `pattern="[A-Za-z]{3}"` for exactly three letters). - **`min`** / **`max`** – Set the minimum and maximum allowed values for numeric or date inputs.
They provide client-side validation, reducing server load and giving immediate feedback.

23. What is the purpose of `placeholder` vs `value` attributes?

- **`placeholder`** – Displays a short hint inside the input (e.g., "Enter your name"). It disappears when the user starts typing and reappears if the field is empty. It is **not submitted** with the form.
- **`value`** – Sets the initial (default) value of the input, which is pre-filled and can be edited. This value **is submitted** with the form if unchanged.
  `placeholder` guides the user; `value` provides actual data.

24. What is the difference between `<button>`, `<input type="button">`, and `<input type="submit">`?

- **`<button>`** – A versatile container that can hold HTML content (text, images, etc.). Its default `type` inside a form is `submit`, causing form submission. It can also be `type="button"` for generic JavaScript buttons.
- **`<input type="button">`** – A simple button with text defined by the `value` attribute. It has no default behavior and is mainly used with JavaScript. It cannot contain HTML.
- **`<input type="submit">`** – A button that submits the form. Its text is set by the `value` attribute. It behaves like `<button type="submit">` but without inner HTML. Use `<button>` for richer content; use `<input type="submit">` for simple form submission.

25. What is the `target` attribute in anchor tags? What is `target="_blank"` and what security concern does it have?

The `target` attribute specifies where to open the linked document. Common values: - `_self` – Same frame (default). - `_blank` – New window or tab. - `_parent` – Parent frame. - `_top` – Full body of the window.

`target="_blank"` opens the link in a new tab.
Security concern: When used without `rel="noopener"` or `rel="noreferrer"`, the new page gains access to the original page's `window.opener` object, which can lead to **tabnabbing** – the new page may redirect the original page to a malicious site.
Fix: Always include `rel="noopener noreferrer"` in the link: `<a href="..." target="_blank" rel="noopener noreferrer">`. This prevents the new page from accessing `window.opener`.

Advanced Level

26. What is the purpose of ARIA attributes in HTML?

ARIA (Accessible Rich Internet Applications) attributes are a set of special HTML attributes designed to improve web accessibility for users with disabilities, particularly those relying on assistive technologies like screen readers. They provide semantic information about elements that native HTML may not convey, such as roles, states, and properties.

Purpose: - **Define roles** – Specify the type of widget or structure (e.g., `role="navigation"`, `role="dialog"`). - **Describe states** – Indicate dynamic changes (e.g., `aria-expanded="true"`, `aria-checked="false"`). - **Provide labels** – Give accessible names when visual labels are insufficient (e.g., `aria-label`, `aria-labelledby`). - **Announce live updates** – Notify assistive tech of content changes (e.g., `aria-live="polite"`).

When to use: Always prefer native HTML semantics first (e.g., `<nav>` instead of `role="navigation"`). Use ARIA only when native elements cannot achieve the needed accessibility, such as custom widgets (e.g., a custom dropdown) or when enhancing complex interactions.

27. How do you make HTML forms accessible?

Creating accessible forms ensures all users, including those with disabilities, can understand, navigate, and submit them successfully. Key practices:

- **Use semantic HTML** – Always wrap form controls in a `<form>` element. Group related controls with `<fieldset>` and provide a `<legend>` to describe the group (e.g., for radio buttons).
- **Associate labels explicitly** – Use `<label>` with the `for` attribute matching the input's `id`, or wrap the input inside the `<label>`. This ensures screen readers announce the label when the input is focused, and clicking the label focuses the input.
- **Provide clear instructions** – Use `<p>` or `<span>` with descriptive text. Link instructions to inputs via `aria-describedby` to have them read by screen readers.
- **Indicate required fields** – Use the `required` attribute and/or visually mark with an asterisk, and include `aria-required="true"` for older browsers.

- **Handle errors accessibly** – Display error messages near the relevant field, use `aria-invalid="true"` on the input, and link the error message with `aria-describedby`.
- **Ensure keyboard navigability** – All form controls should be reachable and operable via keyboard (Tab key, arrow keys for radio/select, etc.).
- **Test with assistive technology** – Use screen readers (NVDA, VoiceOver) to verify announcements and flow.

28. What is the difference between `<script>`, `<script defer>`, and `<script async>`?

These attributes control how and when external JavaScript files are loaded and executed in relation to HTML parsing.

Attribute	Loading	Execution	Order	Use Case
None (normal)	HTML parsing pauses, script fetched, then executed immediately. After execution, parsing resumes.	Blocks parsing	In order (if multiple scripts)	Scripts that must run before page renders or have dependencies on previous scripts.
`defer`	Script fetched in parallel while HTML parses.	After HTML parsing is complete (before `DOMContentLoaded`).	Preserves order	Scripts that need the full DOM and can wait; ideal for most scripts.
`async`	Script fetched in parallel while HTML parses.	As soon as the script is downloaded (may interrupt parsing).	Not guaranteed	Independent scripts (e.g., analytics, ads) that don't rely on other scripts or DOM.

Important: `defer` and `async` are ignored for inline scripts (no `src`). For external scripts, choose based on dependencies.

29. What are HTML entities and when do you need them? Give examples.

HTML entities are special codes that represent characters that either have special meaning in HTML or are difficult to type directly. They begin with an ampersand (`&`) and end with a semicolon (`;`).

Need them when: - Displaying reserved characters that would otherwise be interpreted as HTML code, e.g., < and >. - Inserting characters not available on a standard keyboard, like

copyright symbols, accented letters, or em spaces. - Ensuring correct rendering in all browsers and encodings.

Examples: - `<` → < - `>` → > - `&` → & - `&quot;` → " (used within attribute values) - `&copy;` → © - `&eacute;` → é - ` ` → non-breaking space

30. What is the purpose of <picture> element and how is it different from <img>?

The `<picture>` element is a responsive image container that allows you to provide multiple image sources for different scenarios, such as screen sizes, resolutions, or image formats. It contains zero or more `<source>` elements and one `<img>` as a fallback.

Purpose: - **Art direction** – Serve different image crops for different viewport sizes (e.g., a wide landscape on desktop, a cropped portrait on mobile). - **Format fallbacks** – Offer modern formats like WebP or AVIF for browsers that support them, with JPEG/PNG fallbacks for older browsers. - **Resolution switching** – Can also be used with `srcset` inside `<source>` for density-based selection.

Difference from `<img>`: - `<img>` alone supports `srcset` and `sizes` for resolution switching, but cannot switch based on media queries or provide format fallbacks. - `<picture>` gives more control by allowing media queries and multiple source formats, making it more flexible for advanced responsive needs.

Example:

```
<picture>
  <source srcset="image.avif" type="image/avif">
  <source srcset="image.webp" type="image/webp">
  <img src="image.jpg" alt="Description">
</picture>
```

31. What is the srcset attribute in images?

The `srcset` attribute (used with `<img>` or `<source>` inside `<picture>`) defines a set of image sources with hints to help the browser choose the most appropriate one based on the user's device and viewport.

Syntax: A comma-separated list of image URLs followed by a descriptor: - **Pixel density descriptor (x)** – e.g., `image.jpg 1x, image@2x.jpg 2x`. The browser picks based on device pixel ratio. - **Width descriptor (w)** – e.g., `image-400w.jpg 400w, image-800w.jpg 800w`. The browser uses the `sizes` attribute to calculate the effective display width and picks the best image.

Purpose: Improves performance by loading only the required resolution, saving bandwidth and providing sharper images on high-DPI screens.

Example:

```
<img src="image.jpg"
     srcset="image-400w.jpg 400w, image-800w.jpg 800w, image-1200w.jpg 1200w"
```

```
    sizes="(max-width: 600px) 400px, 800px"
    alt="Description">
```

32. What is the difference between <audio> and <video> elements?

Both are HTML5 elements for embedding media, but they serve different content types:

Feature	`<audio>`	`<video>`
Purpose	Embeds sound content (music, podcasts, sound effects).	Embeds video content (movies, clips, animations).
Visual display	Typically has no visual area; may display a simple audio player (if `controls` attribute is used).	Displays a video area with optional controls.
Supported attributes	`src`, `controls`, `autoplay`, `loop`, `preload`, `muted`.	All of those plus `poster` (image shown before play), `width`, `height`.
Nested `<source>`	Both can use multiple `<source>` elements for format fallbacks (e.g., MP3, OGG).	Same, but for video formats (e.g., MP4, WebM).

Both support JavaScript APIs for custom players.

33. How do you embed iframes and what are the security considerations?

Embedding iframes: Use the `<iframe>` tag with the `src` attribute pointing to the external content:

```
<iframe src="https://example.com" title="Description" width="600"
height="400"></iframe>
```

Additional attributes: `allow`, `sandbox`, `referrerpolicy`, `loading`.

Security considerations: - **Sandboxing** – Apply the `sandbox` attribute to restrict capabilities. By default, it enforces a strict set of restrictions. You can add flags like `allow-scripts`, `allow-forms`, `allow-same-origin` as needed. - **Clickjacking** – Iframes can be used to overlay malicious content on trusted pages. Protect your own pages with the `X-Frame-Options` HTTP header (e.g., `DENY`) or Content Security Policy `frame-ancestors`. - **Permission delegation** – Use the `allow` attribute to control access to features like camera, microphone, or fullscreen. - **Referrer leakage** – Use `referrerpolicy="no-referrer"` or similar to control what referrer information is sent. - **Trust** – Only embed content from trusted sources. Avoid embedding untrusted content without strict sandboxing. - **Performance** – Use `loading="lazy"` for iframes below the fold to improve page load.

34. What is shadow DOM and how does it relate to web components?

Shadow DOM is a browser technology that enables encapsulation of a DOM subtree and its associated CSS styles, isolating them from the main document's DOM. It creates a "shadow boundary" that prevents styles and scripts from leaking in or out.

Key features: - **Encapsulation** – Styles defined inside shadow DOM do not affect the outer page, and vice versa. - **Scoped CSS** – Selectors inside the shadow root are scoped to that component. - **DOM isolation** – Elements inside shadow DOM are not directly accessible via `document.querySelector` (though you can access the shadow root).

Relation to Web Components: Shadow DOM is one of the three core technologies of Web Components (along with Custom Elements and HTML Templates). It allows developers to create reusable, encapsulated custom elements with their own internal structure and styling, without fear of conflict with the rest of the page. For example, a `<custom-tabs>` component can use shadow DOM to hide its internal implementation details from the main document.

35. What is the purpose of `contenteditable` attribute?

The `contenteditable` attribute makes any HTML element editable by the user, transforming it into a rich text editing area. When set to `true`, users can directly modify the element's content via the browser's built-in editing capabilities.

Usage:

```
<div contenteditable="true">Edit this text.</div>
```

Use cases: - **WYSIWYG editors** – Build simple in-page editors. - **Inline editing** – Allow users to quickly update fields without a separate form. - **Interactive widgets** – Create editable lists, notes, or whiteboards.

Important notes: - The edited content is only local; you must capture changes via JavaScript (e.g., `input` event) and send them to the server. - Accessibility: ensure editable regions are announced as such (`role="textbox"` and `aria-multiline` if needed). - Use the `designMode` property on `document` to make the whole page editable (less common).

CSS Interview Questions – Detailed Answers

Basic Level

1. What is CSS and what does it stand for?

CSS stands for **Cascading Style Sheets**. It is a stylesheet language used to describe the presentation and formatting of a document written in HTML (or XML). CSS controls how elements should be rendered on screen, on paper, in speech, or on other media. It enables separation of content (HTML) from presentation, improving accessibility, flexibility, and maintainability.

2. What are the different ways to apply CSS to HTML?

There are three primary ways to apply CSS to HTML: - **Inline CSS** – Using the `style` attribute directly within an HTML element. - **Internal (or Embedded) CSS** – Placing CSS rules inside a `<style>` tag in the HTML `<head>` section. - **External CSS** – Linking to an external `.css` file using the `<link>` tag in the HTML `<head>`.

3. What is the difference between inline, internal, and external CSS?

Method	Definition	Pros	Cons
Inline	Styles applied directly to an element via the `style` attribute.	Quick for testing; overrides other styles.	Mixes content with presentation; hard to maintain; not reusable.
Internal	CSS placed within `<style>` tags in the HTML document.	No extra files; useful for single-page styling.	Increases page size; not reusable across multiple pages.
External	CSS in a separate `.css` file linked via `<link>`.	Separates content from design; cached by browser; reusable across pages; easier maintenance.	Requires additional HTTP request (though caching mitigates this).

Best practice: External CSS is preferred for most projects.

4. What are CSS selectors? List different types.

CSS selectors are patterns used to select and style HTML elements. Types include: - **Universal selector** (`*`) – Targets all elements. - **Type/element selector** (e.g., `div`, `p`, `h1`) - **Class selector** (`.classname`) - **ID selector** (`#idname`) - **Attribute selector** (e.g., `[type="text"]`) - **Descendant combinator** (e.g., `div p`) – selects `p` inside `div` - **Child combinator** (`div > p`) – selects direct child `p` - **Adjacent sibling combinator** (`h1 + p`) – selects `p` immediately after `h1` - **General sibling combinator** (`h1 ~ p`) – selects all `p` after `h1` - **Pseudo-classes** (e.g., `:hover`, `:first-child`) - **Pseudo-elements** (e.g., `::before`, `::first-line`)

5. What is the universal selector (*)?

The universal selector (*) matches every element in the document. It is often used to apply global styles, such as resetting margins and paddings:

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

It has the lowest specificity (0,0,0,0).

6. What are class selectors and id selectors?

- **Class selector (`.classname`)** – Targets elements with a specific class attribute. Classes can be reused on multiple elements.
- **ID selector (`#idname`)** – Targets a single element with a specific id attribute. IDs must be unique within a page.

7. What is the difference between class and id in CSS?

Attribute	Class	ID
Uniqueness	Can be used on multiple elements	Must be unique per page
Specificity	Lower (0,0,1,0)	Higher (0,1,0,0)
JavaScript	Accessed via `getElementsByClassName()`	Accessed via `getElementById()`
Usage	For styling groups of elements	For styling a single, unique element

8. What are pseudo-classes? Give 5 examples.

Pseudo-classes define a special state of an element. They are prefixed with a colon (`:`). Examples: - `:hover` – when mouse hovers over element - `:active` – when element is being activated (clicked) - `:focus` – when element gains focus (e.g., input field) - `:first-child` – first child of its parent - `:nth-child(n)` – nth child of its parent - `:visited` – link that has been visited - `:link` – unvisited link

9. What are pseudo-elements? Give examples.

Pseudo-elements style specific parts of an element. They are prefixed with double colons (`::`). Examples: - `::before` – inserts content before an element's content - `::after` – inserts content after an element's content - `::first-line` – styles the first line of a block-level element - `::first-letter` – styles the first letter of a block-level element - `::selection` – styles portion selected by user

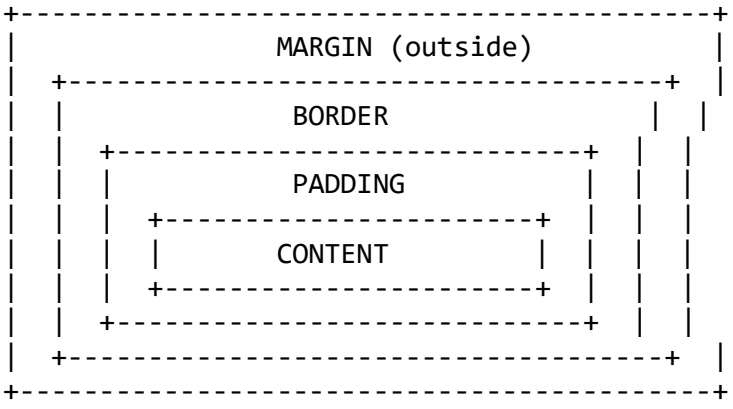
10. What is the difference between `:hover` and `:active`?

- **`:hover`** – Applies when the user's pointer hovers over an element, without activating it.
- **`:active`** – Applies during the moment an element is being activated (e.g., clicked). It typically lasts only as long as the mouse button is held down.

Intermediate Level - Box Model & Layout

11. What is the CSS box model? Explain all components.

The CSS box model is a fundamental concept that describes how every element in HTML is represented as a rectangular box. It consists of four distinct areas:



Components (from inside out): 1. **Content** – The actual content (text, image, etc.). Dimensions set by `width` and `height`. 2. **Padding** – Transparent space between content and border. Clears area inside the box. 3. **Border** – A line surrounding padding (if any) and content. 4. **Margin** – Transparent space outside the border, separating the element from others.

12. What is the difference between margin and padding?

- **Margin** – Creates space **outside** the element's border, between the element and adjacent elements. It is transparent and can collapse with adjacent margins.
- **Padding** – Creates space **inside** the element, between the content and the border. It is part of the element's clickable/background area and is affected by background color.

13. What is margin collapse and when does it occur?

Margin collapse occurs when vertical margins of adjacent block-level elements combine (collapse) into a single margin equal to the larger of the two margins. It happens in three scenarios: 1. **Adjacent siblings** – Bottom margin of first element meets top margin of second. 2. **Parent and first/last child** – If no border/padding separates them, margins can collapse through the parent. 3. **Empty blocks** – Top and bottom margins of an empty block can collapse.

Horizontal margins never collapse.

14. What is `box-sizing: border-box` and why is it useful?

`box-sizing: border-box` changes how an element's total width and height are calculated. With this setting, the `width` and `height` properties include **content, padding, and border** – not just content.

Why it's useful: - Simplifies layout calculations - Prevents unexpected overflow when adding padding/border - Makes responsive design easier - Elements maintain their specified dimensions regardless of padding/border changes

15. What is the default value of `box-sizing`?

The default value is `content-box`.

16. What is the difference between `content-box` and `border-box`?

Aspect	`content-box` (default)	`border-box`
Width calculation	Width = content width only	Width = content + padding + border
Total width	Increases when padding/border added	Stays fixed at specified width
Control	Harder to predict layout	Easier to manage layouts
Example	`width: 200px + padding 20px + border 5px = 250px total`	`width: 200px includes everything`

17. What are the different display property values? Explain each.

Common `display` values: - **`block`** – Element takes full width, starts on new line (e.g., `<div>`, `<p>`). - **`inline`** – Element takes only necessary width, flows with text (e.g., `<span>`, `<a>`). - **`inline-block`** – Flows inline but can have width/height and margins like block. - **`none`** – Element removed from document flow (not visible, no space occupied). - **`flex`** – Creates a flex container for flexible layouts. - **`grid`** – Creates a grid container for two-dimensional layouts. - **`table`** – Behaves like `<table>`. - **`list-item`** – Behaves like `<li>`.

18. What is the difference between `display: none` and `visibility: hidden`?

Property	Effect	Space Occupied
`display: none`	Element completely removed from document flow	No space occupied
`visibility: hidden`	Element becomes invisible but remains in layout	Yes, space is preserved

`display: none` also affects browser rendering and screen reader accessibility, while `visibility: hidden` still occupies space.

19. What is `display: inline-block` used for?

`inline-block` combines features of inline and block elements: - Flows inline with text and other inline elements (doesn't force line breaks) - Can have explicit `width`, `height`, `margin`, and `padding` (like block) - Useful for creating horizontal navigation menus, button groups, or grid-like layouts without floats.

20. What is the difference between block and inline-block elements?

Feature	Block	Inline-Block
Line breaks	Starts on new line, creates line break after	Flows inline, no line breaks
Width/height	Respects width/height	Respects width/height
Default width	100% of parent	Shrink-wraps content
Vertical margins	Work normally	Work normally
Horizontal alignment	Can be centered with `margin: auto`	Can be aligned with `text-align` on parent

---

Intermediate Level - Positioning & Specificity

21. What is CSS specificity and how is it calculated?

CSS specificity is the algorithm browsers use to determine which CSS rule takes precedence when multiple rules target the same element. It is calculated as a four-part value: `(a, b, c, d)`

Calculation method: - **a** – Inline styles (1 if present, else 0) - **b** – Number of ID selectors - **c** – Number of class selectors, attribute selectors, and pseudo-classes - **d** – Number of element selectors and pseudo-elements

The universal selector (*) and combinators (>, +, ~) contribute 0 to specificity.

22. What is the specificity value of inline styles, ids, classes, and elements?

Selector Type	Specificity Value (a,b,c,d)	Example
Inline styles	1,0,0,0	`style="color: red;"`
ID selectors	0,1,0,0	`#header`
Class selectors	0,0,1,0	`.menu`
Attribute selectors	0,0,1,0	`[type="text"]`
Pseudo-classes	0,0,1,0	`:hover`
Element selectors	0,0,0,1	`div, p`
Pseudo-elements	0,0,0,1	`::before`
Universal selector	0,0,0,0	`*`

23. What is the cascade in CSS?

The **cascade** is the algorithm that resolves conflicts when multiple CSS rules apply to the same element. It determines which property value "wins" based on a priority system. The cascade considers:

1. **Origin and importance** – Browser styles < user styles < author styles, with `!important` reversing this order
2. **Specificity** – Higher specificity wins
3. **Source order** – If specificity and origin are equal, later rules override earlier ones

24. What does `!important` do and why should it be avoided?

`!important` is a flag added to a CSS declaration that gives it the highest priority, overriding normal specificity rules.

Why avoid it: - Breaks the natural cascade and specificity rules - Makes debugging difficult - Harder to override later (requires another `!important`) - Encourages poor CSS architecture - Can cause maintenance nightmares in large projects

Better alternatives: Use more specific selectors, refactor CSS, or restructure HTML.

25. What are the different position values in CSS?

- **`static`** – Default; element follows normal document flow. `top`/`left`/etc. have no effect.
- **`relative`** – Positioned relative to its normal position; offsets move it but original space is preserved.
- **`absolute`** – Removed from flow; positioned relative to nearest positioned ancestor (not `static`).
- **`fixed`** – Removed from flow; positioned relative to viewport; stays fixed during scrolling.
- **`sticky`** – Hybrid; behaves like `relative` until scroll threshold, then `fixed` within parent.

26. What is the difference between `position: relative` and `position: absolute`?

Feature	`relative`	`absolute`
Reference point	Its normal position in document flow	Nearest positioned ancestor (not `static`)
Document flow	Stays in flow, space preserved	Removed from flow, no space reserved
Effect on siblings	May overlap but leaves gap	Other elements fill its original space
Use case	Slight adjustments; positioning context for children	Precise placement within container

27. What is `position: fixed` and when is it used?

`position: fixed` positions an element relative to the **viewport**, removing it from normal document flow. The element stays in the same place even when the page is scrolled.

Use cases: - Sticky headers/navigation bars - Floating action buttons - Back-to-top buttons - Fixed sidebars - Modal overlays (sometimes combined with absolute)

28. What is `position: sticky` and how does it work?

`sticky` is a hybrid of `relative` and `fixed`. The element is treated as `relative` until it crosses a specified threshold (e.g., `top: 0`), at which point it becomes `fixed` within its parent container.

How it works: - Requires a threshold (`top`, `left`, etc.) to activate - Stays constrained within its parent container - Useful for section headers that stick while scrolling through content - Not supported in older browsers

29. What is `z-index` and when does it work?

`z-index` controls the stacking order of positioned elements (elements with `position` other than `static`). Higher values appear on top of lower values.

When it works: - Only on positioned elements (`relative`, `absolute`, `fixed`, `sticky`) - Also works on flex and grid items - Creates new stacking contexts

Important: `z-index` only compares elements within the same stacking context.

30. What is a stacking context in CSS?

A **stacking context** is a three-dimensional conceptual grouping of elements along the z-axis. Elements within a stacking context are painted together, and their `z-index` values are compared only within that context.

Created by: - Root element (`<html>`) - Positioned elements with `z-index` value (not `auto`) - Elements with `opacity` < 1 - Elements with `transform`, `filter`, `perspective` properties - Flex/grid containers with `z-index` (not `auto`) - Elements with `isolation: isolate`

Understanding stacking contexts helps prevent unexpected layering issues.

Advanced Level - Units & Responsive

31. What are the different CSS units? Explain px, %, em, rem, vh, vw.

CSS units fall into two categories: **absolute** and **relative**.

Unit	Type	Description
px	Absolute	Pixels; fixed size based on device resolution
%	Relative	Percentage of parent element's size

Unit	Type	Description
em	Relative	Relative to parent element's font size
rem	Relative	Relative to root (<html>) font size
vh	Relative	1% of viewport height
vw	Relative	1% of viewport width

32. What is the difference between em and rem?

Aspect	em	rem
Relative to	Parent element's font size	Root (<html>) font size
Compounding	Can compound (nested elements multiply)	No compounding; consistent across document
Predictability	Less predictable in nested elements	Highly predictable
Use case	Components that should scale with parent	Global spacing, typography that should scale uniformly

Example: If root is 16px and parent is 1.5em (24px), then: - 1.5rem = 24px (root-based) - 1.5em on child = 36px (parent-based compounding)

33. When should you use relative units vs absolute units?

Use relative units (em, rem, %, vh, vw) when: - Building responsive designs - Ensuring accessibility (user font size adjustments respected) - Creating scalable components - Setting typography for better readability - Defining layout proportions

Use absolute units (px) when: - Defining borders (hairlines) - Setting media query breakpoints - Elements that must remain exactly the same size - Print stylesheets (using pt or cm)

Best practice: Prefer relative units for most properties, especially for font sizes and spacing.

34. What are viewport units (vh, vw, vmin, vmax)?

Viewport units are relative to the browser's viewport size:

- **vh** – 1% of viewport height. 100vh = full viewport height.
- **vw** – 1% of viewport width. 100vw = full viewport width.
- **vmin** – 1% of the smaller dimension (viewport width or height)
- **vmax** – 1% of the larger dimension (viewport width or height)

Use cases: - Full-height hero sections (height: 100vh) - Full-width elements (width: 100vw) - Responsive typography that scales with screen size - Square elements that maintain aspect ratio (width: 50vmin; height: 50vmin)

35. What are media queries and how do they work?

Media queries are CSS techniques that apply styles conditionally based on device characteristics (viewport width, height, orientation, resolution, etc.). They are fundamental to responsive web design.

Syntax:

```
@media media-type and (media-feature-rule) {
  /* CSS rules */
}
```

Example:

```
@media screen and (max-width: 768px) {
  body {
    font-size: 14px;
  }
}
```

Common features: `width`, `height`, `orientation`, `resolution`, `hover`, `pointer`

36. What is mobile-first approach in responsive design?

Mobile-first is a design and development strategy where you start styling for the smallest screens first, then progressively enhance for larger screens using `min-width` media queries.

Advantages: - Forces prioritization of essential content - Better performance on mobile devices - Simpler base styles - Graceful degradation handled naturally - Aligns with progressive enhancement philosophy

Example:

```
/* Base styles for mobile */
.container {
  width: 100%;
  padding: 10px;
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    width: 750px;
    margin: 0 auto;
  }
}

/* Desktop and up */
@media (min-width: 1024px) {
  .container {
    width: 980px;
```

```
  }
}
```

37. What is the difference between `min-width` and `max-width` in media queries?

Feature	`min-width`	`max-width`
Meaning	Viewport width **greater than or equal to** value	Viewport width **less than or equal to** value
Approach	Mobile-first (progressive enhancement)	Desktop-first (graceful degradation)
Typical usage	Start small, add styles as screen grows	Start large, override styles as screen shrinks
Example	`@media (min-width: 768px) { ... }` applies at 768px and above	`@media (max-width: 767px) { ... }` applies up to 767px

Both can be combined: `@media (min-width: 768px) and (max-width: 1024px) { ... }`

38. What are CSS variables (custom properties) and how do you use them?

CSS variables (officially CSS custom properties) are entities that store specific values for reuse throughout a document. They enable dynamic styling and easier theme management.

Declaration: Using `--` prefix

```css
:root {
  --primary-color: #3498db;
  --spacing-unit: 8px;
}
```

Usage: Using `var()` function

```css
.button {
  background-color: var(--primary-color);
  margin: var(--spacing-unit);
}
```

Advantages: - Reusable values - Easy theming (change one value, updates everywhere) - Can be manipulated with JavaScript - Cascade-aware and inherit - Can be scoped to specific elements

39. What is the `calc()` function in CSS?

`calc()` is a CSS function that performs calculations to determine property values. It can mix different units.

Syntax:

```css
width: calc(100% - 40px);
height: calc(50vh - 2rem);
padding: calc(1em + 2px);
```

Features: - Supports addition (+), subtraction (-), multiplication (*), division (/) - Can combine different units - Whitespace required around + and - operators - Useful for responsive layouts where exact calculations are needed

Use cases: - Creating fluid layouts with fixed-width sidebars - Responsive typography - Centering elements with dynamic calculations

40. What are CSS preprocessors (Sass, Less)? What advantages do they offer?

CSS preprocessors are scripting languages that extend CSS with programming features, then compile to standard CSS for browsers.

Popular preprocessors: - **Sass/SCSS** (most widely used) - **Less** - **Stylus**

Advantages they offer: - **Variables** – Reusable values (colors, fonts, sizes) - **Nesting** – Write nested selectors mirroring HTML structure - **Mixins** – Reusable blocks of styles - **Functions** – Perform calculations and operations - **Partials and imports** – Modular CSS architecture - **Inheritance** – Share style rules with `@extend` - **Mathematical operations** – Built-in calculation capabilities - **Control directives** – Loops and conditionals - **Maintainability** – Easier to manage large codebases - **DRY code** – Write once, use everywhere

Example (SCSS):

```
$primary-color: #3498db;

@mixin flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}

.container {
  @include flex-center;
  background-color: $primary-color;
}
```

JavaScript Introduction & V8 Engine – Detailed Answers

Basic Level

1. What is JavaScript and what are its key features?

JavaScript is a high-level, dynamic, interpreted (or just-in-time compiled) programming language that conforms to the ECMAScript specification. It is primarily known for adding interactivity to web pages, but it is now widely used on servers (Node.js), desktop applications, mobile apps, and even IoT devices.

Key features: - **Lightweight and interpreted** – Easy to learn and run without a compilation step. - **Dynamic typing** – Variables can hold values of any type without type declarations. - **Prototype-based object orientation** – Objects can inherit directly from other objects. - **First-class functions** – Functions are treated as values; they can be assigned, passed, and returned. - **Single-threaded with event loop** – Executes one operation at a time but handles asynchronous tasks via callbacks, promises, and async/await. - **Multi-paradigm** – Supports procedural, object-oriented, and functional programming styles. - **Cross-platform** – Runs in browsers, servers, and many other environments.

2. Explain the difference between interpreted and compiled languages. Where does JavaScript fit?

- **Interpreted languages** – Source code is executed line by line by an interpreter at runtime. They are generally slower but easier to debug and platform-independent. Examples: Python, Ruby, classic JavaScript.
- **Compiled languages** – Source code is transformed into machine code ahead of time (AOT) by a compiler. The resulting binary runs directly on the hardware, offering faster execution but requiring a separate build step. Examples: C, C++, Go.

JavaScript today is neither purely interpreted nor purely compiled. Modern JavaScript engines (like V8) use **Just-In-Time (JIT) compilation**. Initially, code is interpreted, but frequently executed paths ("hot" code) are compiled to optimized machine code during runtime. This hybrid approach gives JavaScript both portability and good performance.

3. What is the V8 engine and which browsers/platforms use it?

V8 is Google's open-source, high-performance JavaScript engine, written in C++. It compiles JavaScript directly to native machine code (using JIT) and manages memory, garbage collection, and optimizations. V8 is the core engine of: - **Google Chrome** browser - **Chromium-based browsers** (Microsoft Edge, Opera, Brave, etc.) - **Node.js** (server-side JavaScript runtime) - **Deno** (another server-side runtime) - **Electron** (framework for desktop apps)

4. What is Just-In-Time (JIT) compilation?

JIT compilation combines interpretation and ahead-of-time compilation. In JavaScript engines: 1. The source code is first parsed and converted to an intermediate representation (bytecode). 2. An interpreter executes the bytecode, collecting runtime type information. 3. When a piece of code is executed many times (becomes "hot"), the engine sends it to an optimizing compiler that produces highly optimized machine code. 4. If the assumptions made during optimization break (e.g., types change), the engine can "deoptimize" and fall back to interpreted bytecode.

This approach yields the speed of compiled code while maintaining the flexibility of interpretation.

5. Explain the role of the Call Stack in JavaScript execution.

The **call stack** is a last-in-first-out (LIFO) data structure that tracks the execution of functions. Its role: - When a function is called, a new **stack frame** (containing arguments, local variables, and return address) is pushed onto the stack. - When a function returns, its frame is popped off. - The stack ensures that execution returns to the correct place after a function call. - It also enables the tracking of scope chains and helps with error tracing (stack traces).

Because JavaScript is single-threaded, there is only one call stack. Asynchronous operations are handled via the event loop and callback queues, not by additional stacks.

Intermediate Level

6. Describe the V8 engine architecture in detail (Parser, AST, Ignition, TurboFan).

V8's modern architecture (Ignition + TurboFan) consists of several components:

- **Parser** – Reads the source code and produces an **Abstract Syntax Tree (AST)**. It can parse lazily: functions are not fully parsed until they are called, saving memory and startup time.
- **Ignition** – V8's interpreter. It takes the AST and generates platform-independent **bytecode**. This bytecode is executed by Ignition, and during execution, profiling data (type feedback) is collected.
- **TurboFan** – The optimizing compiler. When a function becomes hot, TurboFan uses the collected type feedback to compile the bytecode into highly optimized machine code. It applies advanced optimizations like inlining, constant folding, and hidden class transitions. If assumptions fail, it deoptimizes back to Ignition.
- (Historically, V8 also had Full-codegen and Crankshaft, but they have been replaced by Ignition+TurboFan.)

7. What is the difference between the Memory Heap and Call Stack?

- **Memory Heap** – An unstructured region of memory used for dynamic allocation. Objects, arrays, closures, and functions are stored here. Memory is managed by the garbage collector, which periodically reclaims unused memory.
- **Call Stack** – A structured, ordered region that stores function call frames and primitive values (like numbers and booleans) that are local to functions. It operates in LIFO order and has a fixed size (stack overflow occurs if it exceeds that limit).

Feature	Call Stack	Memory Heap
Purpose	Tracks function calls, stores primitive values and references	Stores objects, arrays, and dynamic data
Structure	LIFO (ordered)	Unstructured (graph of objects)
Size	Fixed, limited	Dynamically grows, managed by GC

Feature	Call Stack	Memory Heap
Lifetime	Frame exists until function returns	Object lives until no references

8. How does the V8 engine optimize JavaScript code for better performance?

V8 employs multiple optimization techniques: - **Inline Caching (IC)** – Caches the results of property lookups based on the object's hidden class, speeding up repeated accesses. - **Hidden classes** – Track object shape to optimize property access. - **JIT compilation** – Compiles hot functions with TurboFan. - **Type feedback** – Collects runtime types to generate specialized code. - **Escape analysis** – Determines if an object's lifetime is limited to a function, allowing allocation on the stack instead of the heap. - **Dead code elimination** – Removes unreachable or unused code. - **Inlining** – Replaces function calls with the function body to reduce overhead. - **Loop peeling** and **hoisting** – Optimize loops by moving invariant code outside.

9. Explain the compilation phases in V8: Parse, Compile, Execute.

1. **Parse** – The source code is tokenized and transformed into an AST. V8 may perform **lazy parsing** for functions that are not immediately invoked, delaying full parsing until needed.
2. **Compile** – The AST is compiled into bytecode by Ignition. For hot functions, TurboFan compiles the bytecode (with type feedback) to optimized machine code.
3. **Execute** – The bytecode (or optimized machine code) is executed. While executing, the engine continues to gather profiling information to guide future optimizations. If an optimized function's assumptions are invalidated, V8 deoptimizes and reverts to bytecode execution.

10. What is the role of Hidden Classes in V8 optimization?

Hidden classes (also called **maps**) are an internal mechanism used by V8 to efficiently represent the shape of objects. In JavaScript, objects can have properties added or removed dynamically, which makes property lookup expensive. Hidden classes solve this by: - Tracking the current set of properties and their offsets. - When two objects have the same hidden class, property access can be optimized using a fixed offset (like in C++ structs). - If a property is added in a different order, a new hidden class is created, and a transition tree links them.

By using hidden classes, V8 can generate efficient inline cache code and avoid dictionary-like lookups, significantly speeding up property access.

Variables (var, let, const) – Detailed Answers

Basic Level

1. What are the different ways to declare variables in JavaScript?

JavaScript provides three keywords for variable declaration: - **`var`** – The original way, function-scoped. - **`let`** – Introduced in ES6, block-scoped, allows reassignment. - **`const`** – Also ES6, block-scoped, creates a read-only reference (immutable binding).

Additionally, variables can be created implicitly by assignment without a keyword (not recommended), and via function/class declarations.

2. What is the difference between var, let, and const?

Feature	`var`	`let`	`const`
Scope	Function scope	Block scope	Block scope
Hoisting	Hoisted and initialized with `undefined`	Hoisted but not initialized (TDZ)	Hoisted but not initialized (TDZ)
Redeclaration	Allowed in same scope	Not allowed in same scope	Not allowed in same scope
Reassignment	Allowed	Allowed	Not allowed
Initialization	Optional (default `undefined`)	Optional (default `undefined`)	**Required** at declaration

3. Can you reassign a variable declared with const?

No, you cannot reassign a `const` variable. Attempting to do so throws a `TypeError`. However, if the variable holds an object or array, the **contents** of that object can be modified (e.g., adding properties, pushing to an array). The binding itself is constant, not the value.

```
const obj = { name: 'John' };
obj.name = 'Jane'; // allowed – object mutated
obj = {}; // TypeError: Assignment to constant variable
```

4. What is the scope of var, let, and const?

- **`var`** – Scoped to the **function** in which it is declared. If declared outside any function, it becomes a global variable (attached to the global object). It ignores block boundaries (`if`, `for`, `{}`).
- **`let` and `const`** – Scoped to the nearest enclosing **block** `{ ... }`, which includes functions, `if`, `for`, `while`, and standalone blocks. They are not visible outside that block.

5. What happens if you declare a variable without var, let, or const?

In **non-strict mode**, an assignment to an undeclared variable creates a property on the global object (implicit global). This can lead to accidental globals and hard-to-debug issues.

```
function test() {
  x = 10; // no keyword – becomes global
}
test();
console.log(x); // 10 (global)
```

In **strict mode** (`"use strict"`), such an assignment throws a `ReferenceError`.

Intermediate Level

6. What is the Temporal Dead Zone (TDZ)?

The **Temporal Dead Zone** is the period between entering a scope (e.g., a block) and the actual declaration of a variable with `let` or `const`. During this time, the variable exists but cannot be accessed in any way; referencing it throws a `ReferenceError`. This prevents the use of variables before their declaration, encouraging cleaner code.

```
{
  // TDZ starts
  console.log(a); // ReferenceError
  let a = 5;      // TDZ ends here
}
```

7. Explain block scope vs function scope with examples.

- **Function scope**: Variables declared with `var` are visible throughout the entire function, regardless of block boundaries.

  ```
  function example() {
    if (true) {
      var x = 10;
    }
    console.log(x); // 10 – accessible outside the if block
  }
  ```

- **Block scope**: Variables declared with `let` or `const` are confined to the block in which they are defined.

  ```
  function example() {
    if (true) {
      let y = 20;
    }
    console.log(y); // ReferenceError: y is not defined
  }
  ```

8. Why is var not recommended in modern JavaScript?

- **Function-scoping can be confusing** – It does not respect block boundaries, leading to unintended variable leakage.
- **Hoisting with `undefined`** – Can mask errors; variables can be used before declaration without throwing an error.
- **Redeclaration allowed** – Accidentally redeclaring a variable in the same scope does not produce a warning.
- **Creates global properties** – When used outside a function, it attaches to the global object.
- **Lack of block scope** – Often causes bugs in loops and conditionals.

Modern best practice favors `let` and `const` for their clearer scoping and error-prevention characteristics.

9. Can you declare a const variable without initialization?

No. A `const` declaration **must** be initialized with a value at the point of declaration. The following is a syntax error:

```
const PI; // SyntaxError: Missing initializer in const declaration
```

10. What is the difference between let and const in terms of reassignment?

- **`let`** – Allows reassigning a new value to the variable.

  ```
  let count = 1;
  count = 2; // OK
  ```

- **`const`** – Does **not** allow reassignment. The variable identifier cannot be bound to a different value.

  ```
  const MAX = 100;
  MAX = 200; // TypeError: Assignment to constant variable
  ```

  However, if the variable holds an object or array, its properties can be mutated; only the binding is immutable.

Advanced Level

11. Output-based: What will be the output?

```
var x = 10;
if (true) {
  var x = 20;
  console.log(x);
}
console.log(x);
```

Explanation: `var` is function-scoped (or globally scoped). There is no block scope, so the `x` inside the `if` refers to the same variable as the outer `x`. The assignment overwrites it.
Output:

```
20
20
```

12. Output-based: What will be the output?

```
let a = 10;
if (true) {
  let a = 20;
  console.log(a);
}
console.log(a);
```

Explanation: `let` is block-scoped. The `a` inside the `if` is a separate variable that shadows the outer `a`. The outer `a` remains unchanged. **Output:**

```
20
10
```

13. Output-based: What will be the output?

```
const arr = [1, 2, 3];
arr.push(4);
console.log(arr);
arr = [5, 6, 7];
console.log(arr);
```

Explanation: `arr` is declared with `const`, so the variable cannot be reassigned. However, `arr.push(4)` mutates the existing array, which is allowed. The reassignment attempt `arr = [5,6,7]` throws a `TypeError`. In practice, the script will stop at the error. **Output:**

```
[1, 2, 3, 4]
TypeError: Assignment to constant variable.
```

(If the error is caught, the second `console.log` is never executed.)

14. Explain variable shadowing with examples.

Variable shadowing occurs when a variable declared in an inner scope has the same name as a variable in an outer scope. The inner variable "shadows" (hides) the outer one, making the outer variable inaccessible within the inner scope.

```
let name = "Alice";
function greet() {
  let name = "Bob"; // shadows the outer 'name'
  console.log("Hello, " + name);
}
greet(); // Hello, Bob
console.log(name); // Alice (outer unaffected)
```

Shadowing works with both `var`, `let`, and `const`, but with `var` it is limited to function scope, while `let/const` respect block scope.

15. What are the best practices for using var, let, and const in production code?

- **Prefer `const` by default** – Use `const` for variables that should not be reassigned. This signals intent and prevents accidental reassignments.
- **Use `let` only when reassignment is necessary** – For loop counters, accumulators, or values that change over time.
- **Avoid `var` entirely** – It has confusing scoping rules and can lead to bugs. If you must support very old environments, consider transpilation.
- **Declare variables at the top of their scope** – Improves readability and avoids TDZ confusion.
- **Use meaningful names** – Avoid single-letter names except in trivial loops.
- **Minimize global variables** – Encapsulate code in functions or modules to prevent pollution of the global namespace.
- **Use block scope to limit variable lifetime** – Declare variables as close as possible to where they are used.

Hoisting in JavaScript – Detailed Answers

Basic Level

1. What is hoisting in JavaScript?

Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their containing scope during the compilation phase, before code execution. This means that regardless of where declarations appear in the code, they are processed first, allowing you to use variables and functions before they appear in the code. However, only the declarations are hoisted—initializations remain in place.

Example:

```
console.log(a); // undefined (not ReferenceError)
var a = 5;
```

The `var a` declaration is hoisted to the top, so the code behaves as if it were:

```
var a;
console.log(a); // undefined
a = 5;
```

2. Which declarations are hoisted in JavaScript?

- **Function declarations** – Entire function body is hoisted, so they can be called before their definition.
- **`var` variables** – Declaration is hoisted, but initialization remains. They are set to `undefined` until the assignment line is reached.

- **`let` and `const` variables** – Declarations are hoisted, but they are not initialized. Accessing them before declaration results in a `ReferenceError` due to the Temporal Dead Zone (TDZ).
- **`class` declarations** – Similar to `let`/`const`: hoisted but not initialized, so accessing them before declaration throws a `ReferenceError`.

3. Are variables declared with let and const hoisted?

Yes, `let` and `const` declarations are **hoisted**. However, they are not initialized with a default value. They enter a **Temporal Dead Zone** from the start of the block until the declaration is encountered. During this period, accessing them throws a `ReferenceError`. This behavior prevents accidental use before declaration.

4. What value is assigned to hoisted var variables before initialization?

Before the line where the `var` variable is initialized, it holds the value `undefined`. The declaration is hoisted, but the assignment stays in place. So until the assignment is executed, the variable exists but is `undefined`.

5. What is the difference between hoisting of var and let/const?

Feature	`var`	`let` / `const`
Hoisting	Declaration hoisted	Declaration hoisted
Initialization	Automatically initialized to `undefined`	Not initialized (TDZ)
Access before declaration	Returns `undefined`	Throws `ReferenceError`
Scope	Function-scoped	Block-scoped

Intermediate Level

6. Explain the creation phase and execution phase in JavaScript.

JavaScript engines process code in two phases:

- **Creation Phase** (also called compilation or parsing phase):
  - The engine scans the code and sets up memory space for variables and functions.
  - For `var` variables, memory is allocated and they are initialized with `undefined`.
  - For `let` and `const`, memory is allocated but they are left uninitialized (TDZ).
  - Function declarations are stored in memory with their entire body.
  - The scope chain and `this` binding are determined.
- **Execution Phase**:

- Code is executed line by line.
- Assignments to variables happen here.
- Functions are invoked, creating new execution contexts (with their own creation and execution phases).

7. What is the Temporal Dead Zone in context of hoisting?

The **Temporal Dead Zone (TDZ)** is the period between entering a scope (like a block) and the actual declaration of a variable with `let` or `const`. During this time, the variable exists in the scope but cannot be accessed in any way. Any attempt to read or write to it throws a `ReferenceError`. This ensures that variables are not used before their declaration, promoting better code quality.

Example:

```
{
  // TDZ starts for 'x'
  console.log(x); // ReferenceError
  let x = 5;      // TDZ ends here
}
```

8. How are function declarations hoisted differently from function expressions?

- **Function declarations** are fully hoisted: both the name and the function body are hoisted. They can be called before the line they appear.
- **Function expressions** (where a function is assigned to a variable) follow the hoisting rules of the variable used. For example:
  - `var func = function() {}` – the variable `func` is hoisted with `undefined`, so calling it before the assignment results in a `TypeError` (because `undefined` is not a function).
  - `let func = function() {}` – the variable is hoisted but uninitialized (TDZ), so accessing it before declaration throws a `ReferenceError`.

9. Are arrow functions hoisted?

Arrow functions are always **function expressions**, not declarations. They follow the hoisting rules of the variable they are assigned to: - If assigned to a `var`, the variable is hoisted as `undefined`; calling it before the assignment throws a `TypeError`. - If assigned to a `let` or `const`, the variable is hoisted but uninitialized; accessing it before the declaration throws a `ReferenceError`.

10. What is the hoisting behavior of class declarations?

`class` declarations are hoisted similarly to `let` and `const`. They are hoisted but not initialized. Therefore, they also have a Temporal Dead Zone. Attempting to use a class before its definition results in a `ReferenceError`.

Example:

```
const obj = new MyClass(); // ReferenceError
class MyClass {}
```

Advanced Level – Output Based

11. Output-based: What will be the output?

```
console.log(a);
var a = 5;
console.log(a);
```

Explanation: The `var a` declaration is hoisted to the top, initialized with `undefined`. The first `console.log` outputs `undefined`. Then `a` is assigned `5`. The second `console.log` outputs `5`. **Output:**

```
undefined
5
```

12. Output-based: What will be the output?

```
console.log(b);
let b = 10;
```

Explanation: `let` declarations are hoisted but remain uninitialized (TDZ). Accessing `b` before its declaration throws a `ReferenceError`. The code stops execution at that point. **Output:**

```
ReferenceError: Cannot access 'b' before initialization
```

13. Output-based: What will be the output?

```
test();
function test() {
  console.log("Function Declaration");
}
```

Explanation: Function declarations are fully hoisted. The function can be called before its definition. So `test()` executes and prints the string. **Output:**

```
Function Declaration
```

14. Output-based: What will be the output?

```
test();
var test = function() {
  console.log("Function Expression");
}
```

Explanation: The variable `test` is hoisted (as `var`) with value `undefined`. When `test()` is called, `test` is still `undefined`, causing a `TypeError`. The assignment never happens because an error occurs first. **Output:**

```
TypeError: test is not a function
```

15. Output-based: What will be the output?

```
var x = 10;
function foo() {
  console.log(x);
  var x = 20;
  console.log(x);
}
foo();
```

Explanation: Inside `foo`, `var x` is hoisted to the top of the function scope. The outer `x` (global) is **shadowed** by the local `x`. The first `console.log(x)` accesses the hoisted but not yet initialized local `x`, which is `undefined`. Then `x = 20` assigns it, so the second log outputs `20`. **Output:**

```
undefined
20
```

16. Output-based: What will be the output?

```
function test() {
  console.log(a);
  console.log(b);
  var a = 10;
  let b = 20;
}
test();
```

Explanation: - `var a` is hoisted to the top of the function and initialized with `undefined`. First log: `undefined`. - `let b` is hoisted but uninitialized (TDZ). Before the `let` declaration, `b` is in TDZ. Accessing it throws a `ReferenceError`. The error occurs at the second `console.log`, so the function stops there. No further execution. **Output:**

```
undefined
ReferenceError: Cannot access 'b' before initialization
```

17. Output-based: What will be the output?

```
var a = 10;
{
  console.log(a);
  let a = 20;
}
```

Explanation: The outer `var a` is in the global scope. Inside the block, `let a` creates a new variable that shadows the outer one. However, due to TDZ, the `let a` is hoisted but uninitialized within the block. The `console.log(a)` inside the block tries to access the block-scoped `a` **before** its declaration, causing a `ReferenceError`. The outer `a` is not accessible because the block variable shadows it. **Output:**

```
ReferenceError: Cannot access 'a' before initialization
```

18. Output-based: What will be the output?

```
console.log(typeof myFunc);
var myFunc = function() {};
console.log(typeof myFunc);
```

Explanation: The `var myFunc` declaration is hoisted with `undefined`. The first `typeof myFunc` evaluates to `"undefined"`. After the assignment, `myFunc` becomes a function, so the second `typeof` returns `"function"`. **Output:**

```
undefined
function
```

19. Output-based: What will be the output?

```
foo();
var foo = function() {
  console.log("First");
}
foo();
function foo() {
  console.log("Second");
}
foo();
```

Explanation: This involves both function declaration and variable hoisting. The function declaration `function foo() {...}` is hoisted first, completely. Then the `var foo` declaration is hoisted, but since `foo` already exists, it does not overwrite the function (variable declarations do not overwrite function declarations). So initially, `foo` refers to the function that logs `"Second"`.

The code execution: 1. `foo()` – calls the function (still the declared one), prints `"Second"`. 2. `var foo = function() {...}` – now the variable assignment overwrites `foo` with the new function expression. 3. `foo()` – now `foo` is the expression function, prints `"First"`. 4. `foo()` – again the expression function, prints `"First"`.

Thus output:

```
Second
First
First
```

20. Explain the hoisting behavior in the following code and why it behaves that way:

```
var x = 1;
function outer() {
  console.log(x);
  var x = 2;
  function inner() {
    console.log(x);
    var x = 3;
    console.log(x);
  }
```

```
  inner();
  console.log(x);
}
outer();
```

Step-by-step hoisting analysis:

- Global scope: `var x = 1` (global).
- Function `outer` is declared (hoisted globally).
- Inside `outer`, `var x` is hoisted to the top of the function scope (local `x` shadows global). Initially `undefined`.
- Inside `outer`, function `inner` is declared (hoisted to the top of `outer`).

Execution of `outer()`:

1. `console.log(x)` – refers to the local `x` (because `var x` is hoisted inside `outer`). It is currently `undefined`. Output: `undefined`.
2. `var x = 2` – assigns local `x` to `2`.
3. `inner()` called:
   - Inside `inner`, `var x` is hoisted to the top of `inner`. This `x` shadows the outer `x` from `outer`. Initially `undefined`.
   - `console.log(x)` – the local `x` of `inner` is `undefined`. Output: `undefined`.
   - `var x = 3` – assigns local `x` to `3`.
   - `console.log(x)` – now local `x` is `3`. Output: `3`.
4. After `inner` returns, `console.log(x)` inside `outer` – now refers to the local `x` of `outer`, which is `2`. Output: `2`.

Final output:

```
undefined
undefined
3
2
```

Why it behaves that way: Each function creates its own scope. Variable declarations with `var` are hoisted to the top of their function, shadowing outer variables. Inside each function, the local variable exists (as `undefined`) from the start, overriding any outer variable of the same name. The assignments happen later.

Functions in JavaScript – Detailed Answers

Basic Level

1. What are the different ways to define a function in JavaScript?

JavaScript offers several ways to define functions:

- **Function Declaration** – Uses the `function` keyword followed by a name.

```
function greet(name) {
  return `Hello, ${name}`;
}
```

- **Function Expression** – A function assigned to a variable. The function can be named or anonymous.

```
const greet = function(name) {
  return `Hello, ${name}`;
};
```

- **Arrow Function** – A concise syntax using `=>`, introduced in ES6.

```
const greet = (name) => `Hello, ${name}`;
```

- **Method Definition in Objects** – Functions defined as object properties.

```
const obj = {
  greet(name) {
    return `Hello, ${name}`;
  }
};
```

- **Function Constructor** – Using `new Function()` (not recommended due to security and performance issues).

```
const greet = new Function('name', 'return "Hello, " + name');
```

- **Generator Function** – Uses `function*` and can be paused/resumed.

```
function* generator() {
  yield 1;
}
```

- **Async Function** – Declared with `async function` for promises.

```
async function fetchData() { ... }
```

2. What is the difference between function declaration and function expression?

Feature	Function Declaration	Function Expression
Syntax	`function name() {}`	`const name = function() {};` or `let name = function() {};`
Hoisting	Fully hoisted; can be called before definition	Not hoisted; only variable declaration is hoisted (if `var`), but the assignment is not; if `let`/`const`, the variable is in TDZ
Name	Must have a name	Can be anonymous or named

Feature	Function Declaration	Function Expression
Use	Suitable for standalone functions	Useful for assigning functions dynamically, callbacks, or conditional definitions

3. What are arrow functions and how are they different from regular functions?

Arrow functions are a compact alternative to function expressions, introduced in ES6. Syntax: `(params) => expression` or `(params) => { statements }`.

Differences from regular functions: - **No own `this`** – Arrow functions inherit `this` from the enclosing lexical scope. They do not have their own `this`, so `this` cannot be changed with `call`, `apply`, or `bind`. - **No `arguments` object** – They do not have their own `arguments` object; use rest parameters instead. - **Cannot be used as constructors** – Calling with `new` throws an error. - **No `prototype` property** – Arrow functions do not have a `prototype`. - **Cannot be used as methods** (if you need to access object via `this`) – They are not suitable for object methods that need dynamic `this`. - **Cannot have duplicate named parameters** (even in non-strict mode).

4. What is the difference between parameters and arguments?

- **Parameters** – Variables listed in the function definition that act as placeholders for the values the function will receive.
- **Arguments** – The actual values passed to the function when it is invoked.

```
function add(a, b) {  // a and b are parameters
  return a + b;
}
add(5, 3);            // 5 and 3 are arguments
```

5. What are default parameters in JavaScript functions?

Default parameters allow you to initialize function parameters with default values if no argument is provided or if `undefined` is passed. They are defined using an equals sign `=` in the parameter list.

```
function greet(name = "Guest") {
  return `Hello, ${name}`;
}
console.log(greet());        // Hello, Guest
console.log(greet("John"));  // Hello, John
```

Intermediate Level

6. What are rest parameters and how are they used?

Rest parameters allow a function to accept an indefinite number of arguments as an array. They are denoted by three dots `...` before the last parameter. The rest parameter must be the last parameter.

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
```

7. What is the spread operator and how does it differ from rest parameters?

- **Spread operator** (`...`) expands an iterable (like an array) into individual elements. It is used in function calls, array literals, or object literals.

  ```
  const arr = [1, 2, 3];
  console.log(Math.max(...arr)); // 3
  ```

- **Rest parameters** collect multiple elements into a single array.

Key difference: Spread *expands* an array into elements; rest *collects* elements into an array. The context determines which behavior occurs.

8. What is an IIFE (Immediately Invoked Function Expression) and why is it used?

An **IIFE** is a function that is defined and executed immediately after creation. Syntax:

```
(function() {
  // code
})();
// or
(() => {
  // code
})();
```

Purposes: - Create a private scope to avoid polluting the global namespace. - Implement the module pattern before ES6 modules. - Execute asynchronous code immediately without leaving temporary variables in global scope. - Used in older JavaScript to encapsulate code.

9. What is a callback function?

A **callback function** is a function passed as an argument to another function, which is then invoked inside the outer function to complete some action. Callbacks are fundamental for asynchronous operations (e.g., event handlers, `setTimeout`, AJAX calls) and higher-order functions (e.g., `map`, `filter`).

```
function fetchData(callback) {
  setTimeout(() => {
    callback("Data received");
  }, 1000);
}
fetchData((message) => console.log(message));
```

10. What is function hoisting and how does it differ from variable hoisting?

- **Function hoisting** – For function declarations, the entire function definition is hoisted to the top of its scope. This allows calling the function before its declaration.

- **Variable hoisting** – For variables declared with `var`, only the declaration is hoisted, not the initialization. The variable is set to `undefined` until the assignment line. For `let/const`, hoisting occurs but the variable is in a Temporal Dead Zone until the declaration.

Example:

```
console.log(foo()); // Works: "foo"
function foo() { return "foo"; }

console.log(bar);   // undefined (var hoisting)
var bar = "bar";
```

Advanced Level

11. Explain the difference between function declarations, function expressions, and arrow functions in terms of hoisting.

- **Function declarations** – Fully hoisted. The entire function (name and body) is moved to the top of its scope. They can be used before they appear.
- **Function expressions** – Follow the hoisting rules of the variable they are assigned to:
  - `var` function expression: the variable declaration is hoisted with value `undefined`, so calling it before the assignment throws a `TypeError` (because `undefined` is not a function).
  - `let/const` function expression: the variable is hoisted but uninitialized (TDZ), so accessing it before declaration throws a `ReferenceError`.
- **Arrow functions** – They are always expressions. Like function expressions, they follow the hoisting rules of the variable they are assigned to. There is no arrow function *declaration*.

12. What are higher-order functions? Give examples.

A **higher-order function** is a function that either: - Takes one or more functions as arguments, or - Returns a function as its result.

Examples: - Array methods: `map()`, `filter()`, `reduce()`, `forEach()`. - `setTimeout()` and event listeners. - Custom functions like `function createMultiplier(factor) { return (x) => x * factor; }`.

13. How do arrow functions handle the `this` keyword differently from regular functions?

- **Regular functions** – Have their own `this` value, determined by how they are called (dynamic `this`). The value can be set via call, apply, bind, or the calling context.
- **Arrow functions** – Do not have their own `this`. They inherit `this` from the enclosing lexical scope (the `this` value of the surrounding non-arrow function). This makes them ideal for callbacks where you want to preserve the outer `this`, e.g., inside event handlers or `setTimeout` within a class method. They cannot be used as

constructors, and `this` cannot be changed with `call/apply/bind` (though arguments can be passed, the `this` is ignored).

14. What is the `arguments` object and does it exist in arrow functions?

The **`arguments` object** is an array-like object available inside regular functions that contains the values of the arguments passed to that function. It is not a real array but has a `length` property and can be accessed by index. It exists only in regular functions (including function declarations and expressions), **not in arrow functions**. In arrow functions, use rest parameters instead.

```
function regular() {
  console.log(arguments[0]); // 1
}
regular(1, 2, 3);

const arrow = () => {
  console.log(arguments); // ReferenceError: arguments is not defined
};
```

15. Explain function currying with an example.

Currying is a functional programming technique where a function with multiple arguments is transformed into a sequence of nested functions, each taking a single argument. The curried function returns a new function for each argument until all arguments are supplied.

Example:

```
// Normal function
function multiply(a, b, c) {
  return a * b * c;
}

// Curried version
function curryMultiply(a) {
  return function(b) {
    return function(c) {
      return a * b * c;
    };
  };
}
// Using arrow functions:
const curryMultiply = a => b => c => a * b * c;

console.log(curryMultiply(2)(3)(4)); // 24
```

Currying allows partial application, creating specialized functions:

```
const multiplyBy2 = curryMultiply(2);
const multiplyBy2And3 = multiplyBy2(3);
console.log(multiplyBy2And3(4)); // 24
```

Output Based

16. Output-based: What will be the output?

```
function sum(a, b = 5) {
  return a + b;
}
console.log(sum(10));
console.log(sum(10, 20));
```

Explanation: The first call passes only one argument, so `b` defaults to `5`. `10 + 5 = 15`. The second call passes both arguments, so `b` is `20`. `10 + 20 = 30`. **Output:**

```
15
30
```

17. Output-based: What will be the output?

```
function test(...args) {
  console.log(args);
  console.log(typeof args);
}
test(1, 2, 3, 4, 5);
```

Explanation: The rest parameter `...args` collects all arguments into an array. So `args` is `[1, 2, 3, 4, 5]`. `typeof` an array is `"object"`. **Output:**

```
[1, 2, 3, 4, 5]
object
```

18. Output-based: What will be the output?

```
(function() {
  var a = 10;
  console.log(a);
})();
console.log(a);
```

Explanation: This is an IIFE. Inside the IIFE, `a` is declared with `var` and logged as `10`. Outside, `a` is not defined because `var` is function-scoped to the IIFE. The second `console.log(a)` throws a `ReferenceError` (or, if run in non-strict mode, might throw an error). In practice, the error stops execution after the IIFE logs `10`. So the output would be `10` followed by an error. But if we consider the output of the successful parts, it's `10` then error. The question likely expects just the first line and then an error. **Output (in console):**

```
10
ReferenceError: a is not defined
```

19. Output-based: What will be the output?

```
const multiply = (a, b) => a * b;
console.log(multiply(5, 3));
```

Explanation: Arrow function with implicit return of `a * b`. `5 * 3 = 15`. **Output:**

```
15
```

20. Output-based: What will be the output?

```
function outer() {
  var x = 10;
  function inner() {
    console.log(x);
  }
  return inner;
}
var func = outer();
func();
```

Explanation: `outer()` returns the `inner` function. This returned function retains access to the scope of `outer` (closure). When `func()` is called, it logs the `x` from that closure, which is `10`. **Output:**

```
10
```

Higher-Order Functions – Detailed Answers

Basic Level

1. What is a higher-order function in JavaScript?

A **higher-order function** is a function that either: - Takes one or more functions as arguments, or - Returns a function as its result.

In JavaScript, functions are first-class citizens, which makes higher-order functions possible. They are a key concept in functional programming and enable powerful patterns like abstraction, composition, and code reuse.

Example:

```
// Takes a function as argument
function greet(name, formatter) {
  return formatter(name);
}
console.log(greet("John", (name) => `Hello, ${name}!`)); // Hello, John!

// Returns a function
function createMultiplier(factor) {
  return (x) => x * factor;
```

```
}
const double = createMultiplier(2);
console.log(double(5)); // 10
```

2. What does it mean that functions are first-class citizens in JavaScript?

First-class citizens means that functions in JavaScript are treated like any other value. They can be: - Assigned to variables or object properties. - Passed as arguments to other functions. - Returned from other functions. - Stored in data structures (arrays, objects).

This capability is what enables higher-order functions. For example, you can store a function in a variable and then pass it around.

3. How does JavaScript enable higher-order functions?

JavaScript enables higher-order functions because of its treatment of functions as first-class objects. Functions have a type (`function`) and can be manipulated like any other object. The language provides syntax and runtime support for passing functions as arguments and returning them. Additionally, closures (functions retaining access to their lexical scope) allow returned functions to remember the environment in which they were created, which is essential for many higher-order patterns.

4. Give 3 examples of built-in higher-order functions in JavaScript.

1. **`Array.prototype.map()`** – Takes a callback function and returns a new array with the results of calling that function on every element.
2. **`Array.prototype.filter()`** – Takes a predicate function and returns a new array with elements that pass the test.
3. **`Array.prototype.reduce()`** – Takes a reducer function and an optional initial value, and accumulates a single result.

All of these accept a function as an argument, making them higher-order functions.

5. Can a function return another function? Give an example.

Yes, a function can return another function. This is a common pattern for creating function factories or closures.

Example:

```
function makeGreeter(greeting) {
  return function(name) {
    console.log(`${greeting}, ${name}!`);
  };
}
const sayHello = makeGreeter("Hello");
sayHello("Alice"); // Hello, Alice!
const sayHi = makeGreeter("Hi");
sayHi("Bob"); // Hi, Bob!
```

The returned function “remembers” the `greeting` variable from its outer scope (closure).

Intermediate Level

6. Explain how `map()` is a higher-order function.

`map()` is a higher-order function because it accepts a callback function as its argument. This callback is applied to each element of the array, and `map()` returns a new array containing the transformed values. By abstracting the iteration logic, `map()` allows you to focus on the transformation itself.

```
const numbers = [1, 2, 3];
const doubled = numbers.map(n => n * 2); // callback passed as argument
console.log(doubled); // [2, 4, 6]
```

7. Explain how `filter()` is a higher-order function.

`filter()` is a higher-order function because it takes a predicate function (a function that returns a boolean) as an argument. It iterates over the array, applies the predicate to each element, and returns a new array containing only the elements for which the predicate returned `true`. The callback defines the filtering logic.

```
const numbers = [1, 2, 3, 4, 5];
const evens = numbers.filter(n => n % 2 === 0); // callback passed
console.log(evens); // [2, 4]
```

8. Explain how `reduce()` is a higher-order function.

`reduce()` is a higher-order function that takes a reducer function (and optionally an initial value) as arguments. The reducer is called for each element, accumulating a result. The accumulator logic is entirely defined by the provided callback.

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((acc, curr) => acc + curr, 0); // callback passed
console.log(sum); // 10
```

9. What is the difference between a higher-order function and a regular function?

A **regular function** performs a specific task using only its own logic and data. It may accept primitive values or objects as arguments and may return a value, but it does not operate on functions as data.

A **higher-order function** distinguishes itself by either accepting a function as an argument or returning a function (or both). This capability allows higher-order functions to abstract over actions, enabling more flexible and reusable code patterns.

10. Create a higher-order function that takes a function as an argument and executes it twice.

```
function doTwice(fn) {
  fn();
  fn();
}
```

```
doTwice(() => console.log("Hello"));
// Output:
// Hello
// Hello
```

This higher-order function abstracts the “do something twice” pattern. The specific action is supplied as a callback.

Advanced Level

11. What is function composition? How do higher-order functions enable it?

Function composition is the process of combining two or more functions to produce a new function. For example, given functions `f` and `g`, composition creates `h(x) = f(g(x))`. In JavaScript, higher-order functions make composition natural because functions can be passed around and returned.

Example:

```
const compose = (f, g) => (x) => f(g(x));
const toUpperCase = (str) => str.toUpperCase();
const exclaim = (str) => `${str}!`;
const shout = compose(exclaim, toUpperCase);
console.log(shout("hello")); // "HELLO!"
```

`compose` is a higher-order function that takes two functions and returns a new function, enabling the composition of behavior.

12. What are the benefits of using higher-order functions?

- **Abstraction** – Hide repetitive patterns (e.g., iteration) and let you focus on the core logic.
- **Reusability** – Create flexible functions that can be used in many contexts by supplying different callbacks.
- **Modularity** – Break down complex tasks into smaller, composable units.
- **Readability** – Code becomes more declarative and expressive (e.g., using `map`, `filter` instead of explicit loops).
- **Functional composition** – Easily combine simple functions to build complex behavior.
- **Testability** – Small, focused functions are easier to test in isolation.

13. How do higher-order functions improve code reusability?

Higher-order functions allow you to write generic algorithms that can be customized with different callbacks. For example, a `sort` function can accept a comparison function to handle different data types or sorting orders. This means you write the sorting logic once and reuse it with any comparison strategy. Similarly, array methods like `map`, `filter`, and `reduce` are reusable across all arrays and operations.

14. What is the relationship between higher-order functions and functional programming?

Functional programming is a paradigm that emphasizes the use of pure functions, immutability, and function composition. Higher-order functions are a cornerstone of functional programming because they enable: - **Abstraction over actions** – Instead of writing loops, you use `map`, `filter`, etc. - **Function composition** – Building complex operations by combining simple functions. - **Currying and partial application** – Creating specialized functions from general ones. - **Callbacks and asynchronous patterns** – Often used in functional reactive programming.

Without higher-order functions, many functional programming techniques would be impossible or cumbersome.

15. Create a higher-order function that returns a function (closure example).

```
function makeCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

`makeCounter` is a higher-order function that returns a function. The returned function "closes over" the `count` variable, preserving its state between calls. This is a classic example of a closure created by a higher-order function.

Output Based

16. Output-based: What will be the output?

```
function higherOrder(fn) {
  fn();
  fn();
}
higherOrder(function() { console.log("Hello"); });
```

Explanation: The function `higherOrder` accepts a function `fn` and calls it twice. The passed callback logs `"Hello"` each time. **Output:**

```
Hello
Hello
```

17. Output-based: What will be the output?

```
function multiplier(factor) {
  return function(number) {
```

```
    return number * factor;
  };
}
const double = multiplier(2);
console.log(double(5));
```

Explanation: `multiplier` is a higher-order function that returns a new function. `double` is the returned function that multiplies its argument by `2`. Calling `double(5)` gives `10`. **Output:**

```
10
```

18. Output-based: What will be the output?

```
const numbers = [1, 2, 3];
const doubled = numbers.map(function(n) { return n * 2; });
console.log(doubled);
```

Explanation: `map` applies the callback to each element, returning a new array with the results. Each number is multiplied by 2. **Output:**

```
[2, 4, 6]
```

19. Output-based: What will be the output?

```
function operate(a, b, operation) {
  return operation(a, b);
}
const result = operate(5, 3, function(x, y) { return x + y; });
console.log(result);
```

Explanation: `operate` is a higher-order function that calls the passed `operation` with `a` and `b`. The callback adds its arguments, so `5 + 3 = 8`. **Output:**

```
8
```

20. Output-based: What will be the output?

```
function createCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
const counter = createCounter();
console.log(counter());
console.log(counter());
console.log(counter());
```

Explanation: `createCounter` returns a function that increments and returns a private `count` variable (closure). Each call increments the same `count`. So first call returns `1`, second `2`, third `3`. **Output:**

1
2
3

Callback Functions – Detailed Answers

Basic Level

1. What is a callback function in JavaScript?

A **callback function** is a function passed as an argument to another function, which is then invoked inside the outer function to complete some routine or action. Callbacks are fundamental for handling asynchronous operations (like timers, events, or network requests) and for customizing the behavior of higher-order functions (e.g., `map`, `filter`).

2. Why are callback functions used?

- **Asynchronous programming** – Execute code after a task completes without blocking.
- **Reusability** – Write generic functions that can be customized by passing different callbacks.
- **Event handling** – Respond to user interactions (clicks, key presses).
- **Functional programming** – Enable array methods like `map`, `reduce`, and `filter`.
- **Separation of concerns** – Decouple "what to do" from "when to do it."

3. How do you pass a function as a callback?

You pass the function by name (without parentheses) or as an inline function expression. The receiving function will call it later, often with arguments.

```
function greet(name) { console.log("Hello " + name); }
setTimeout(greet, 1000, "John"); // greet passed as callback

// anonymous callback
setTimeout(function() { console.log("Done"); }, 1000);
```

4. What is the difference between a callback function and a regular function?

There is no difference in definition. A **callback** is simply a function that is *used* as an argument to another function. A **regular function** may be called directly; a callback is called by the function it was passed to, possibly later (asynchronously) or repeatedly.

5. Give 3 examples of functions that accept callbacks.

- **`setTimeout(callback, delay)`** – Executes callback after a delay.
- **`Array.prototype.map(callback)`** – Applies callback to each array element.
- **`addEventListener(event, callback)`** – Calls callback when event occurs.

Intermediate Level

6. What is the difference between synchronous and asynchronous callbacks?

- **Synchronous callbacks** – Executed immediately inside the outer function, before it returns. They block further execution. Examples: `map`, `filter`, `forEach`.
- **Asynchronous callbacks** – Executed later, after the outer function returns, usually when an event or task completes. They are placed in the event queue. Examples: `setTimeout`, event handlers, `fs.readFile`.

7. How do synchronous callbacks work? Give an example with `map()`.

`map()` iterates over the array and calls the callback synchronously for each element, using its return value to build a new array. The program waits for each callback to finish before moving to the next element.

```
[1, 2, 3].map(n => n * 2); // callback runs immediately for each item
```

8. How do asynchronous callbacks work? Give an example with `setTimeout()`.

`setTimeout()` starts a timer and returns immediately. When the timer expires, the callback is placed in the task queue. The event loop executes it when the call stack is empty.

```
console.log("Start");
setTimeout(() => console.log("Inside"), 1000);
console.log("End");
// Output: Start, End, Inside
```

9. How do you pass arguments to a callback function?

You can wrap the callback in an anonymous function, use `.bind()`, or rely on built-in functions that accept additional arguments (e.g., `setTimeout(fn, delay, arg1, arg2)`). For custom functions, the outer function can pass arguments when calling the callback.

```
function process(callback, data) { callback(data); }
process(console.log, 42); // logs 42
```

10. What is an anonymous callback vs a named callback?

- **Anonymous callback** – A function without a name, defined inline. Not reusable elsewhere.
- **Named callback** – A function with a name, defined separately and passed by name. Easier to debug (stack trace shows the name) and can be reused.

11. What are the advantages of using callback functions?

- Non-blocking asynchronous operations.
- Customizable behavior for generic functions.
- Reusability and modularity.
- Enables functional programming patterns.
- Simplifies event-driven programming.

12. How are callbacks used in event handling?

Event listeners are callback functions that the browser calls when a specified event occurs (click, load, etc.). This allows the program to react to user interactions without polling.

```
button.addEventListener("click", (event) => console.log("Clicked"));
```

13. How are callbacks used in array methods like `forEach()`, `map()`, `filter()`?

These methods accept a callback that defines the operation to be performed on each element. The method handles the iteration; the callback supplies the specific logic.

```
[1, 2, 3].forEach(num => console.log(num));
```

14. What is callback hell and why is it a problem?

Callback hell (pyramid of doom) refers to deeply nested callbacks, typically from multiple dependent asynchronous operations. It makes code hard to read, maintain, and debug, and leads to poor error handling and tight coupling.

15. How can you avoid callback hell?

- Use **named functions** to flatten the structure.
- Switch to **Promises** and `.then()` chaining.
- Use **async/await** for a synchronous-looking flow.
- Leverage control flow libraries (e.g., `async.js`).

Advanced Level

16. How do callbacks enable asynchronous programming in JavaScript?

JavaScript is single-threaded. Without callbacks, long operations (like file I/O) would block execution. Callbacks allow the runtime to continue processing other code and later invoke the callback when the operation finishes, enabling non-blocking concurrency.

17. What is the relationship between callbacks and the event loop?

Asynchronous callbacks are not executed immediately; they are queued in the task queue. The **event loop** continuously checks if the call stack is empty, then dequeues a callback and pushes it onto the stack for execution. This mechanism allows JavaScript to handle many operations without multi-threading.

18. How do error-first callbacks work in Node.js?

In Node.js, the standard pattern for asynchronous callbacks passes an error as the first argument. If the operation succeeds, `err` is `null` or `undefined`; subsequent arguments hold the result. This convention ensures consistent error handling.

```
fs.readFile('file.txt', (err, data) => {
  if (err) throw err;
```

```
  console.log(data);
});
```

19. What are the limitations of callbacks compared to Promises?

- **Inversion of control** – The calling code loses control; error handling is manual.
- **Callback hell** – Deep nesting for sequential operations.
- **No standard error propagation** – Must check errors in every callback.
- **Difficult to combine multiple asynchronous results** – Requires manual coordination.
- **No built-in cancellation or timeout mechanisms**.

20. Output-based: What will be the output?

```
function processData(data, callback) {
  console.log("Processing:", data);
  callback(data * 2);
}
processData(5, function(result) {
  console.log("Result:", result);
});
```

Explanation: `processData` logs "Processing: 5", then calls the callback with `10`, which logs "Result: 10". **Output:**

```
Processing: 5
Result: 10
```

Objects – Detailed Answers

Basic Level

1. What are objects in JavaScript and how do you create them?

Objects are collections of key-value pairs (properties). Keys are strings (or Symbols) and values can be any type, including functions (methods). Creation methods include: - **Object literal**: `const obj = { name: "John", age: 30 };` - **`new Object()`**: `const obj = new Object(); obj.name = "John";` - **Constructor function**: `function Person(name) { this.name = name; } const p = new Person("John");` - **`Object.create()`**: `const obj = Object.create(proto);` - **ES6 class**: `class Person { constructor(name) { this.name = name; } }`

2. What is the difference between dot notation and bracket notation for accessing object properties?

- **Dot notation** (`obj.prop`): Requires property name to be a valid identifier (no spaces, not starting with a digit, not a reserved word). It is static – you must know the property name at coding time.

- **Bracket notation** (`obj["prop"]`): Accepts any string expression, including variables. Useful for dynamic property names or names with special characters.

3. How do you add, modify, and delete properties from an object?

- **Add/Modify**: Use assignment: `obj.newProp = value;` or `obj["existing"] = newValue;`.
- **Delete**: Use the `delete` operator: `delete obj.propName;`

4. What is the `delete` operator and how does it work?

`delete` removes a property from an object. It returns `true` if successful (or if the property was non-configurable, it returns `false` in strict mode). It affects only the object's own properties, not prototype properties, and does not directly free memory.

5. What is the `this` keyword in object methods?

Inside a method, `this` refers to the object that the method was called on. It allows access to other properties of the same object.

```
const person = {
  name: "Alice",
  greet() { console.log("Hi, " + this.name); }
};
person.greet(); // Hi, Alice
```

Note: `this` is dynamic; if the method is detached, `this` may become the global object or `undefined`.

Intermediate Level

6. What are the different ways to create objects in JavaScript (literal, constructor, Object.create)?

- **Literal**: `{ prop: value }` – simplest, most common.
- **Constructor function**: `function Person(name) { this.name = name; }` – creates objects with a shared prototype.
- **`Object.create(proto)`**: Creates a new object with the specified prototype – useful for prototypal inheritance.
- **ES6 class**: Syntactic sugar over constructor functions.

7. What is object destructuring and how does it work?

Object destructuring unpacks properties into variables.

```
const person = { name: "John", age: 30 };
const { name, age } = person;
console.log(name); // John
```

You can rename variables, set defaults, and nest destructuring.

8. Explain the difference between shallow copy and deep copy.

- **Shallow copy**: Creates a new object with the same top-level properties. Nested objects are shared (references copied). Methods: `Object.assign()`, spread `{...obj}`.
- **Deep copy**: Recursively copies all nested objects, creating independent duplicates. Methods: `JSON.parse(JSON.stringify(obj))` (limited), `structuredClone()`, or custom recursive functions.

9. What are `Object.keys()`, `Object.values()`, and `Object.entries()`?

- `Object.keys(obj)` – Returns an array of own enumerable property names.
- `Object.values(obj)` – Returns an array of own enumerable property values.
- `Object.entries(obj)` – Returns an array of `[key, value]` pairs.

10. How do you check if a property exists in an object?

- `'prop' in obj` – Checks own and inherited properties.
- `obj.hasOwnProperty('prop')` – Checks only own properties.
- `obj.prop !== undefined` – Checks if value is not `undefined` (but property could exist with value `undefined`).

Advanced Level - Output Based

11. Output-based: What will be the output?

```
const obj = { name: "John", age: 30 };
delete obj.age;
console.log(obj);
```

Explanation: `delete` removes the `age` property. The object now only contains `name`.
Output:

```
{ name: "John" }
```

12. Output-based: What will be the output?

```
const a = {};
const b = { key: "b" };
const c = { key: "c" };
a[b] = 123;
a[c] = 456;
console.log(a[b]);
```

Explanation: Object keys are converted to strings. Both `b` and `c` become `"[object Object]"`, so the second assignment overwrites the first. `a[b]` reads the same key and returns `456`. **Output:**

```
456
```

13. Output-based: What will be the output?

```
const obj = { a: 1, b: 2, c: 3 };
const { a, ...rest } = obj;
console.log(a);
console.log(rest);
```

Explanation: Destructuring extracts `a` into a variable; the rest (`b`, `c`) are collected into `rest` via the rest syntax. **Output:**

```
1
{ b: 2, c: 3 }
```

14. Output-based: What will be the output?

```
const person = {
  name: "Alice",
  greet: function() {
    console.log(this.name);
  }
};
const greet = person.greet;
greet();
```

Explanation: The method is detached from its object. Called as a plain function, `this` defaults to the global object (non-strict) or `undefined` (strict). In non-strict mode, it logs `undefined`. In strict mode, it throws an error. Typically, the answer expected is `undefined`. **Output (non-strict):**

```
undefined
```

15. Output-based: What will be the output?

```
const obj1 = { a: 1, b: { c: 2 } };
const obj2 = { ...obj1 };
obj2.b.c = 3;
console.log(obj1.b.c);
console.log(obj2.b.c);
```

Explanation: Spread creates a shallow copy. `obj1.b` and `obj2.b` reference the same nested object. Changing `obj2.b.c` modifies the shared object, so both logs show `3`. **Output:**

```
3
3
```

C.10 Arrays & Array Methods – Detailed Answers

Basic Level

1. What is an array and how do you create it in JavaScript?

An **array** is an ordered collection of values (elements) that can be of any data type. Arrays are zero-indexed and have a `length` property.

Creation methods: - **Array literal:** `const arr = [1, 2, 3];` - **`new Array()` constructor:** `const arr = new Array(1, 2, 3);` (if a single numeric argument, it sets the length: `new Array(5)` creates a sparse array of length 5) - **`Array.of()`:** `const arr = Array.of(1, 2, 3);` (always creates an array with the given elements) - **`Array.from()`:** converts iterables or array-like objects into an array:

```
const arr = Array.from('hello'); // ['h','e','l','l','o']
```

2. What is the difference between `map()` and `forEach()`?

- **`map()`** creates a **new array** with the results of calling a provided function on every element. It is used for transformation and does not modify the original array.
- **`forEach()`** executes a provided function once for each array element; it returns `undefined`. It is used for side effects (e.g., logging, modifying external variables) and does not return a new array.

3. What does the `filter()` method do?

`filter()` creates a **new array** containing all elements that pass a test implemented by a provided function (the function returns `true`). It does not modify the original array.

4. What is the `reduce()` method and how does it work?

`reduce()` executes a **reducer function** on each element, resulting in a single output value. It takes two arguments: - a reducer function with parameters `accumulator` (the accumulated result) and `currentValue` (the current element) - an optional initial value for the accumulator.

If no initial value is provided, the first element becomes the accumulator and iteration starts from the second element.

Example:

```
[1, 2, 3].reduce((sum, n) => sum + n, 0); // 6
```

5. What is the difference between `push()` and `pop()` methods?

- **`push()`** adds one or more elements to the **end** of an array and returns the new length.
- **`pop()`** removes the **last** element from an array and returns that element.

Both methods **modify the original array**.

Intermediate Level

6. Explain the `map()` method with an example. Does it modify the original array?

`map()` transforms each element and returns a new array. The original array remains unchanged.

Example:

```
const arr = [1, 2, 3];
const doubled = arr.map(x => x * 2);
console.log(doubled); // [2, 4, 6]
console.log(arr);     // [1, 2, 3] (unchanged)
```

7. Explain the `filter()` method with an example. What does it return?

`filter()` returns a new array containing only the elements for which the callback returns a truthy value.

Example:

```
const arr = [1, 2, 3, 4, 5];
const evens = arr.filter(x => x % 2 === 0);
console.log(evens); // [2, 4]
```

8. Explain the `reduce()` method in detail. What are accumulator and current value?

`reduce(callback, initialValue)` processes each element: - **`accumulator`** – the value returned by the previous iteration (or the `initialValue` for the first call). - **`currentValue`** – the current element being processed.

The callback returns the new accumulator. After the last element, the final accumulator is returned.

Example without initial value:

```
[1, 2, 3].reduce((acc, curr) => acc + curr);
// first call: acc=1, curr=2 → 3
// second call: acc=3, curr=3 → 6
```

9. How do you chain `map()`, `filter()`, and `reduce()` methods?

Since each method returns an array (except `reduce` which returns a single value), you can chain them sequentially.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const result = numbers
  .map(x => x * 2)          // [2,4,6,8,10]
  .filter(x => x > 5)       // [6,8,10]
  .reduce((sum, x) => sum + x, 0); // 24
console.log(result); // 24
```

10. What is the difference between `map()` and `reduce()`?

- **`map()`** transforms each element independently and returns an array of the same length.
- **`reduce()`** aggregates all elements into a single value (which can be of any type – number, string, object, etc.).

Advanced Level

11. Write a polyfill for the `map()` method.

```
Array.prototype.myMap = function(callback, thisArg) {
  if (this == null) throw new TypeError('this is null or undefined');
  if (typeof callback !== 'function') throw new TypeError(callback + ' is not
a function');
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) { // handle sparse arrays
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};
```

12. Write a polyfill for the `filter()` method.

```
Array.prototype.myFilter = function(callback, thisArg) {
  if (this == null) throw new TypeError('this is null or undefined');
  if (typeof callback !== 'function') throw new TypeError(callback + ' is not
a function');
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) {
      if (callback.call(thisArg, this[i], i, this)) {
        result.push(this[i]);
      }
    }
  }
  return result;
};
```

13. Write a polyfill for the `reduce()` method.

```
Array.prototype.myReduce = function(callback, initialValue) {
  if (this == null) throw new TypeError('this is null or undefined');
  if (typeof callback !== 'function') throw new TypeError(callback + ' is not
a function');
  const arr = this;
  let accumulator = initialValue;
  let startIndex = 0;

  if (accumulator === undefined) {
```

```
    if (arr.length === 0) throw new TypeError('Reduce of empty array with no
initial value');
    // Find first non-empty element
    while (startIndex < arr.length && !(startIndex in arr)) startIndex++;
    if (startIndex >= arr.length) throw new TypeError('Reduce of empty array
with no initial value');
    accumulator = arr[startIndex];
    startIndex++;
  }

  for (let i = startIndex; i < arr.length; i++) {
    if (i in arr) {
      accumulator = callback(accumulator, arr[i], i, arr);
    }
  }
  return accumulator;
};
```

14. What is the time complexity of `map()`, `filter()`, and `reduce()`?

All three methods have **O(n)** time complexity, where `n` is the length of the array, because they iterate through each element once (assuming the callback itself is O(1)). If the callback performs additional work, the complexity increases accordingly.

15. How do array methods handle empty elements?

Array methods like `map`, `filter`, and `reduce` **skip empty slots** (holes) in sparse arrays. They do not invoke the callback for indices that have never been assigned. For example:

```
const sparse = [1, , 3];
const mapped = sparse.map(x => x * 2);
console.log(mapped); // [2, empty, 6]
```

Output Based

16. Output-based: What will be the output?

```
const arr = [1, 2, 3, 4, 5];
const result = arr.map(x => x * 2);
console.log(result);
console.log(arr);
```

Output:

```
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]
```

Explanation: `map()` returns a new array; the original `arr` remains unchanged.

17. Output-based: What will be the output?

```
const arr = [1, 2, 3, 4, 5];
const result = arr.filter(x => x % 2 === 0);
console.log(result);
```

Output: `[2, 4]` **Explanation:** `filter()` returns only the elements that satisfy the condition (even numbers).

18. Output-based: What will be the output?

```
const arr = [1, 2, 3, 4, 5];
const result = arr.reduce((acc, curr) => acc + curr, 0);
console.log(result);
```

Output: `15` **Explanation:** The reducer sums all elements.

19. Output-based: What will be the output?

```
const arr = [1, 2, 3, 4, 5];
const result = arr
  .map(x => x * 2)
  .filter(x => x > 5)
  .reduce((acc, curr) => acc + curr, 0);
console.log(result);
```

Step-by-step: - `map` → `[2, 4, 6, 8, 10]` - `filter ( >5 )` → `[6, 8, 10]` - `reduce` sum → `6 + 8 + 10 = 24` **Output:** `24`

20. Output-based: What will be the output?

```
const users = [
  { name: "John", age: 25 },
  { name: "Jane", age: 30 },
  { name: "Bob", age: 25 }
];
const result = users.reduce((acc, curr) => {
  acc[curr.age] = (acc[curr.age] || 0) + 1;
  return acc;
}, {});
console.log(result);
```

Explanation: `reduce` builds an object counting occurrences of each age. Starting with an empty object, for each user it increments the count for that age. **Output:** `{ "25": 2, "30": 1 }`

C.11 Destructuring – Detailed Answers

Basic Level

1. What is destructuring in JavaScript?

Destructuring is a syntax that allows unpacking values from arrays or properties from objects into distinct variables. It provides a concise way to extract data.

2. What is array destructuring and how does it work?

Array destructuring uses square brackets `[]` on the left-hand side to match array elements by position.

Example:

```
const [a, b] = [1, 2];
console.log(a); // 1
console.log(b); // 2
```

3. What is object destructuring and how does it work?

Object destructuring uses curly braces `{}` to match property names.

Example:

```
const {name, age} = {name: 'John', age: 30};
console.log(name); // John
console.log(age);  // 30
```

4. How do you assign default values in destructuring?

Use `=` after the variable name to provide a default if the value is `undefined`.

Array example:

```
const [a = 10] = []; // a = 10
```

Object example:

```
const {name = 'Guest'} = {}; // name = 'Guest'
```

5. What is the rest operator in destructuring?

The rest operator `...` collects remaining elements (array) or properties (object) into a new array/object.

Array rest:

```
const [first, ...rest] = [1, 2, 3, 4];
console.log(first); // 1
console.log(rest);  // [2, 3, 4]
```

Object rest:

```
const {a, ...others} = {a: 1, b: 2, c: 3};
console.log(others); // {b: 2, c: 3}
```

Intermediate & Advanced Level

6. How do you rename variables during object destructuring?

Use `property: newName` syntax.

Example:

```
const {name: userName} = {name: 'John'};
console.log(userName); // John
```

7. What is nested destructuring? Provide an example.

Nested destructuring follows the structure of nested objects/arrays.

Example:

```
const data = {
  user: {
    name: 'Alice',
    address: { city: 'NYC' }
  }
};
const { user: { address: { city } } } = data;
console.log(city); // 'NYC'
```

8. How do you swap two variables using destructuring?

```
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // 2, 1
```

9. How do you use destructuring in function parameters?

Destructure objects or arrays directly in the parameter list.

Example:

```
function greet({name, age}) {
  console.log(`Hello ${name}, age ${age}`);
}
greet({name: 'John', age: 30});
```

10. Output-based: What will be the output?

```
const [a, b, ...rest] = [1, 2, 3, 4, 5];
console.log(a);
```

```
console.log(b);
console.log(rest);
```

Output:

```
1
2
[3, 4, 5]
```

Explanation: a takes the first element, `b` the second, and `rest` collects the remaining into an array.

C.12 Data Types & Data Comparison – Detailed Answers

Basic Level

1. What are the primitive data types in JavaScript?

Primitive types are: `string`, `number`, `boolean`, `null`, `undefined`, `symbol` (ES6), and `bigint` (ES2020). They are immutable and stored by value.

2. What are the reference data types in JavaScript?

Reference types (objects) include: `Object`, `Array`, `Function`, `Date`, `RegExp`, `Map`, `Set`, etc. They are stored by reference and can be mutated.

3. What is the `typeof` operator and how is it used?

`typeof` returns a string indicating the type of the operand.

Examples:

```
typeof 42              // "number"
typeof "hello"         // "string"
typeof true            // "boolean"
typeof undefined       // "undefined"
typeof null            // "object" (historical bug)
typeof {}              // "object"
typeof []              // "object"
typeof function(){}    // "function"
```

4. What is the difference between `null` and `undefined`?

- **`undefined`** means a variable has been declared but not assigned a value, or a function returns nothing.
- **`null`** is an intentional assignment representing "no value" or "empty".
- Loose equality: `null == undefined` → `true`; strict equality: `null === undefined` → `false`.
- `typeof null` returns `"object"` (a known quirk).

5. What is NaN and how do you check for it?

- **`NaN`** (Not-a-Number) is a numeric value representing an undefined or unrepresentable result (e.g., `0/0`, `parseInt('abc')`). It is the only value that is **not equal to itself**.
- Check using `isNaN(value)` (coerces to number) or `Number.isNaN(value)` (no coercion, true only if value is `NaN`).

Intermediate & Advanced Level

6. What is type coercion in JavaScript? Explain implicit and explicit coercion.

Type coercion is automatic or manual conversion of values from one type to another. - **Implicit coercion** happens automatically (e.g., `'5' - 3` → `2`, `'5' + 3` → `'53'`). - **Explicit coercion** is done manually using functions like `Number()`, `String()`, `Boolean()`, or operators like `+` for number conversion.

7. What is the difference between == and ===?

- `==` (loose equality) performs **type coercion** if the operands are of different types.
- `===` (strict equality) checks equality **without type conversion**; both value and type must be the same.
- Always prefer `===` to avoid unexpected coercion.

8. What are truthy and falsy values in JavaScript? List all falsy values.

- **Falsy values** evaluate to `false` in boolean contexts: `false`, `0`, `-0`, `0n` (BigInt zero), `""` (empty string), `null`, `undefined`, `NaN`.
- **Truthy values** are all values that are not falsy (e.g., `"hello"`, `42`, `[]`, `{}`, `true`).

9. Output-based: What will be the output?

```
console.log(typeof null);        // "object"
console.log(typeof undefined);   // "undefined"
console.log(typeof NaN);         // "number"
console.log(typeof []);          // "object"
console.log(typeof {});          // "object"
```

10. Output-based: What will be the output?

```
console.log(1 == "1");        // true  (coercion)
console.log(1 === "1");       // false (different types)
console.log(null == undefined); // true
console.log(null === undefined);// false
console.log(0 == false);      // true  (false coerces to 0)
console.log(0 === false);     // false
```

C.13 Bonus Section: Tricky Interview Questions – Detailed Answers

1. Output-based: What will be the output?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

Output: After 1 second, prints `3` three times. **Explanation:** `var` is function-scoped, not block-scoped. The loop completes and `i` becomes `3` before any `setTimeout` runs. All callbacks share the same `i` and log the final value.

2. Output-based: What will be the output?

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
```

Output: After 1 second, prints `0, 1, 2`. **Explanation:** `let` creates a new binding for each iteration (block scope), so each callback captures the value of `i` at that iteration.

3. Output-based: What will be the output?

```
const arr = [1, 2, 3];
arr[10] = 10;
console.log(arr.length); // 11
console.log(arr[5]);     // undefined
```

Explanation: Setting an index beyond length expands the array. Unset indices are empty slots, but accessing them returns `undefined`. Length is max index+1.

4. Output-based: What will be the output?

```
console.log([] + []);      // ""
console.log([] + {});      // "[object Object]"
console.log({} + []);      // 0   (interpreted as block + [])
console.log({} + {});      // NaN (block + {})
```

Explanation: - `[] + []` → both arrays convert to strings: `"" + "" = ""`. - `[] + {}` → `"" + "[object Object]" = "[object Object]"`. - `{} + []` – in a statement, `{}` is a block, then `+ []` unary plus converts array to number → `0`. - `{} + {}` – block followed by `+ {}` where object converts to `NaN`.

5. Output-based: What will be the output?

```
console.log(+"10");        // 10
console.log(+"abc");       // NaN
console.log(!!"");         // false
console.log(!!"hello");    // true
```

Explanation: Unary plus converts to number; `!!` converts to boolean.

6. Output-based: What will be the output?

```
const obj = {
  a: 1,
```

```
  b: 2,
  a: 3
};
console.log(obj);
```

Output: `{ a: 3, b: 2 }` **Explanation:** Later property definitions overwrite earlier ones with the same key.

7. Output-based: What will be the output?

```
let a = [1, 2, 3];
let b = a;
b.push(4);
console.log(a); // [1,2,3,4]
console.log(b); // [1,2,3,4]
```

Explanation: Both variables reference the same array, so mutation affects both.

8. Output-based: What will be the output?

```
function test() {
  console.log(arguments);
}
test(1, 2, 3);
```

Output: `[Arguments] { '0': 1, '1': 2, '2': 3 }` (an array-like object, not a true array).

9. Output-based: What will be the output?

```
const arr = [1, 2, 3];
const [a, b, c, d = 10] = arr;
console.log(d); // 10
```

Explanation: Destructuring with default; `d` gets the default because there is no fourth element.

10. Output-based: What will be the output?

```
const obj = { x: 1, y: 2 };
const { x: a, y: b } = obj;
console.log(a); // 1
console.log(b); // 2
console.log(x); // ReferenceError: x is not defined
```

Explanation: Variables `a` and `b` are created; `x` is not defined outside the destructuring.

11. Output-based: What will be the output?

```
const nums = [1, 2, 3];
nums[10] = 10;
const doubled = nums.map(x => x * 2);
console.log(doubled);
```

Output: `[2, 4, 6, empty × 7, 20]` (or similar representation). **Explanation:** `map` skips empty slots; indices 0-2 produce values, index 10 produces `20`, indices 3-9 remain empty.

12. Output-based: What will be the output?

```
function outer() {
  let x = 10;
  return function inner() {
    console.log(x);
    let x = 20;
  };
}
outer()();
```

Output: `ReferenceError: Cannot access 'x' before initialization` **Explanation:** Inside `inner`, `let x` is hoisted but in TDZ until its declaration. The `console.log` tries to access the local `x` before initialization, causing a ReferenceError. The outer `x` is shadowed.

13. Output-based: What will be the output?

```
console.log(typeof typeof 1);
```

Output: `"string"` **Explanation:** `typeof 1` returns `"number"`. Then `typeof "number"` returns `"string"`.

14. Output-based: What will be the output?

```
const arr = [1, 2, 3];
const [a, , c] = arr;
console.log(a, c);
```

Output: `1 3` **Explanation:** The second element is skipped using an empty slot.

15. Output-based: What will be the output?

```
function test(a, b = a * 2) {
  console.log(a, b);
}
test(5);     // 5, 10
test(5, 10); // 5, 10
```

Explanation: Default parameters can refer to previous parameters. First call: `a=5`, `b` defaults to `5*2=10`. Second call: explicit `b=10`.
